



第1章 什么是验证

验证不是一个测试平台，也不是一系列测试平台的集合。验证是一个证明设计思路如何实现的过程。验证贯穿于我们的日常生活中：支票簿的结算、品尝食物、在地图上寻找用标记标出的特定目标，这些都是验证的过程。

本章将从重要性和代价的角度介绍验证的基本概念，以及如何确认验证的内容。我们将讨论不同的验证方法之间的差异、测试和验证之间的差异。也将介绍验证在设计复用中的重要性，以及验证复用时所面临的挑战。

1.1 什么是测试平台

术语“测试平台”(Testbench)通常指一段仿真代码，用来为设计产生特定的输入序列，也用来观测设计输出的响应。测试平台常常采用VHDL、Verilog、e或OpenVera语言编写，有时也会调用一些外部的数据文件或C函数。

图1.1是一个测试平台和待测设计(DUV)之间相互关系的示意图。测试平台为设计提供输入信号，并监视设计的输出信号。注意，这个系统是一个完全封闭的系统：系统没有来自外部的输入信号，也没有输出信号。测试平台是系统的控制核心。验证的任务就是确定产生什么样的输入模式，以及期望的设计输出。

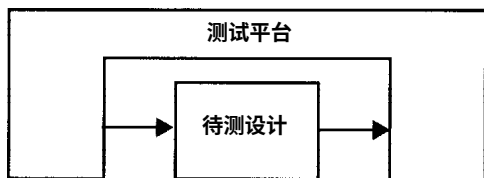


图 1.1 一种常见的测试平台和待测设计的结构

1.2 验证的重要性

大多数书籍关注语法、语义和语言的RTL子集

典型的VHDL和Verilog书籍的大多数章节都在介绍语言的语法和语义，通常也会用两三章的篇幅介绍可综合性编码风格或寄存器传输级(RTL)语言子集。



这些书籍通常只用一章的内容来介绍测试平台。但是一章的内容不足以解释清楚与测试平台有关的知识，并且这些解释常常是非常简单的。几乎所有的书籍都把测试平台用到的技术局限于产生同步方式的激励矢量，用波形浏览工具检测输出信号。大多数书籍介绍语言的文件输入功能，把更多的内容安排在语法和语义方面。

这些书籍中大部分的内容是在讲述如何编写可综合的 VHDL 或 Verilog 代码，而不是强调如何编写验证功能正确性的测试平台，因此读者会得出前者比后者更重要的结论。事实上，这与所有的硬件设计队伍的结论相悖。

验证工作占设计工作量的70%

在这个设计规模达数百万门 ASIC、可复用的知识产权（IP）和系统芯片（SoC）的时代，验证占到了设计工作量的70%。设计团队应该配备能够正确描述验证需求的工程师和专门进行验证的工程师。验证工程师的数量甚至应该是 RTL 设计人员的两倍。

验证是必不可少的

由于验证的工作量巨大，以及缺乏合格的硬件设计和验证工程师，因此几乎所有项目的验证都不能及时完成就不足为奇了。事实上，我们经常在设计完成后才考虑验证工作。当设计进度受到影响时，问题就会产生。这也是现在的新工具和方法学把验证作为研究目标的原因，它们试图通过并行化、更高层次的抽象和自动化来缩短验证需要的时间。

并行化可以缩短验证所需的时间

如果验证工作可以并行进行，那么就可以使用更多的资源来缩短验证所需的时间。这就好比在地上挖一个坑，可以让很多拿着铁锹的工人一起来挖。为了让验证工作得以并行进行，必须让编写代码、调试、验证以及设计的实现过程都并行化。

抽象可以缩短验证所需的时间

采用更高层次的抽象使我们可以不关心底层的细节，从而提高工作效率，这就好像在上面的例子里使用挖土机来挖坑。

抽象减少了对底层细节的控制

高层次的抽象通常伴随着对底层控制的减少，因此必须仔细选择。高层次的抽象要求额外的训练以理解抽象的机制及其后果。

使用挖土机挖坑会遇到同样的失去控制的问题：工人不再直接和泥土打交道，而是操作操作杆和踏板。挖掘的工作更快，但精度降低了，并且只有受过训练的工人才能工作。验证工作可以在更高的抽象层次完成，例如事务级、总线周期级甚至更高的层次，而不是在底层直接和 0、1 打交道。



自动化可以缩短验证所需的时间

自动化可以让我们在机器自主地、快速地完成任务的同时做别的事情，得到可预测的结果。自动化需要编写定义好输入和输出的标准进程。并不是所有的过程都可以自动化，这就好像使用通用的机械不可能挖各种不同形状、尺寸、深度、位置和土质的坑一样。

验证面临着同样的问题。由于要验证各种不同的功能、接口、协议和变换，现在的技术无法采用一种通用的自动化解决方案来进行验证，而只能将某些验证过程自动化，特别是在一些专业的应用领域。这就好像挖沟机可以自动挖出放置管道和电缆的浅沟。用来自动化各种验证过程的工具正在不断出现，例如，有些工具可以从高层次抽象的规范自动产生总线功能模型。

随机化可以作为一种自动化工具

在一些特殊领域，随机化可以用来模拟自动化。通过构造一个随机发生器来产生在特定范围内的有效输入，就有可能自动产生所有感兴趣的输入状态。例如，打扫一个池塘底部的过程可以由以下自动化过程实现：用一个扫帚在池底部自动地随机移动，池的四壁对移动的范围加以限制。几个小时后，可能只剩下角落和几小块地方需要手工清扫了。要取得同样的结果，这种自动化的过程需要更多的计算时间，但它是完全自动的，因此可以让重要的资源去实现其他关键的任务。这个过程可以通过同时运行数个随机发生器并行进行^①。它们可以通宵运行，从而增加了总的生产时间。

1.3 恢复模型

恢复模型（reconvergence model）是验证过程的一种概念表述，人们使用它来精确地解释验证的对象是什么。

你知道到底在验证什么吗

必须回答的最重要的问题是：你正在验证什么？验证的目的是保证某种变换的结果和预期的结果一致。例如，对支票簿结算的目的是确认所有的交易都被准确地记录下来，并且所记录的结余反映了总的可用资金。

验证是通过各种方法核对设计规范和输出结果一致性的过程

图1.2表明了一种变换的验证只能通过另一条有公共起点的恢复路径完成。这里提到的变换可以是任何具有输入和输出的过程。设计规范的RTL编码、插入扫描链、将RTL代码综合成门级网单，以及门级网单的版图设计都是硬件设计中的变换。验证是检查变换的结果和变换的起点是否一致的过程。如果变换和验证没有共同的起点，那么就不存在验证。

^① 优化这些并行进行的过程以减少重复次数是另一个问题。



图 1.2 验证中的恢复路径

可以用图 1.3 所示的支票簿例子来解释恢复模型。变换和验证的公共起点是支票簿中记录的上个月的结余。变换的过程就是在一个月的周期内开出支票、收取支票和还款。验证就是检查支票簿上最后记录的结余是否和本月银行的对账单一致。

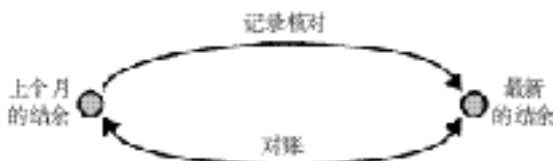


图 1.3 结算支票簿是一个验证的过程

1.4 人为因素

如果端到端的变换过程不是自动完成的，那么就必须由人（或小组）对设计规范加以解释，然后完成变换。RTL 编码就是一个这样的例子。设计小组对书面的设计规范加以解释，并完成他们认为功能正确的、可综合的 HDL 代码。通常每位工程师都要自己验证这些代码是真正“功能正确的”。

根据自己的理解而不是设计规范去验证设计文件

图 1.4 展示了上述情况的恢复路径模型。如果由同一个人对设计规范加以解释，编写 RTL 代码，并完成 RTL 代码的验证，那么验证和变换的共同起点将是“所做的解释”，而不是设计规范。

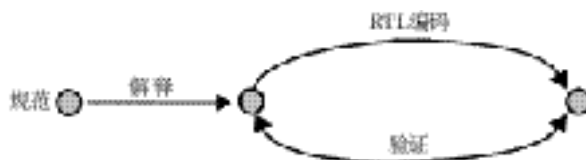


图 1.4 不确定状态下的恢复路径

在这种情况下，验证只检查设计是否准确地表述了设计人员对规范的理解。验证无法检查出设计人员对规范的错误理解。

任何人的介入都会产生不确定性和不可重复性。以下方法可以减少人为错误出现的可能性：自动化、防差错技术或冗余。

1.4.1 自动化

消除人为的错误

自动化显然可以消除人为的错误。虽然自动化的过程没有人的介入,但并不是所有的过程都可自动化,特别是一些没有很好定义、要求人的智慧和创造性的过程,例如硬件的设计。

1.4.2 防差错技术

确保人的介入简单安全

另一种防止人为错误的办法是把人的介入简化成简单、安全的步骤,人的介入仅仅是为了获得预期输出所要求的特定序列或步骤。这种方法在全面质量管理中称为“防差错技术”(Poka-Yoke),它离全自动的过程只有一步之遥。和自动化方法一样,它需要仔细定义的过程和标准的变换步骤。验证过程至今仍然是一种技术,并没有预定的步骤。

1.4.3 冗余

让两个人相互检查对方的工作

最后一种消除人为错误的办法是冗余。它是最简单的方法,也是代价最高的方法。冗余要求每个变换资源都有一个备份。每个由人完成的变换都由另一个人独立验证,或者由两个人分别完成变换,并比较是否产生了相同或等价的结果。这种方法常用在要求高可靠性的场合,例如航空或空间系统中。在工业界的某些场合,重新设计并替换有缺陷产品的代价比冗余的代价更高,例如ASIC设计,这时也采用冗余法。

应该由不同的人负责验证

图1.5显示了恢复路径模型,其中使用了冗余方法来防止对模棱两可的设计规范的错误理解。在硬件设计的过程中,变换就是根据设计规范编写RTL代码,这意味着应该由不同的设计人员负责验证。

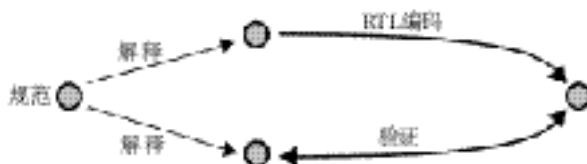


图 1.5 冗余可以在不确定的情况下保证验证的准确性

1.5 验证的对象是什么

选择公共起点和恢复点来确定验证的对象，通常这些选择由验证所使用的工具决定。知道这些点位于什么阶段对于理解正在验证哪种变换非常重要。形式验证、模型检查、功能验证和规则检查具有不同的公共起点和恢复点，因此验证的对象也不同。

1.5.1 形式验证

形式验证并不能消除对编写测试平台的需要

最初，形式验证很容易被误解。不了解形式验证的工程师会把它想像成一种可以从数学角度验证设计正确性的工具，因此就不需要编写测试平台了。只要理解了什么是形式验证恢复路径的终点，就可以理解形式验证的对象是什么。

形式验证的应用可以分为两大类：等效性检查和模型检查。

1.5.2 等效性检查

等效性检查对两个模型进行了比较

图 1.6 是等效性检查的恢复路径模型示意图。形式验证从数学的角度证明了变换过程的起点和输出在逻辑上是等效的，因此变换保持了原来的功能。

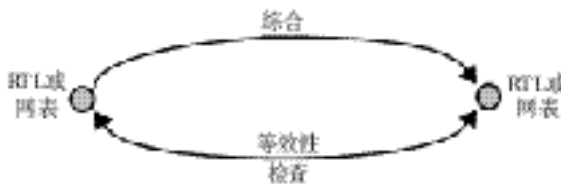


图 1.6 等效性检查的路径

两个网表的比较

在大多数情况下，等效性检查用来比较两个网表，以保证网表的后处理没有改变电路的功能，其中的后处理包括插入扫描链、时钟树综合或手动修改网表等^①。

等效性检查可以检测出综合软件的缺陷

等效性检查的另一种常见的用法是验证网表和原始 RTL 代码的一致性。如果完全信任综合工具，这种验证就没有必要了。然而，综合工具是一个巨大的软件系统，它依赖于算法的正确性和库的信息。事实证明它容易产生错误。等效性检查可以确保综合软件的正确性。在极少数情况下，也使用等效性检查来验证 RTL 代码和相应门级设计的一致性。

^① 文本编辑器仍然是最伟大的设计工具！

有时我们也使用等效性检查来检验两个 RTL 设计在逻辑上是否一致。等效性检查可以用来避免运行冗长的仿真，例如，为获得更好的综合效果对源代码做一些小的非功能性修改，或者把设计从一种 HDL 语言翻译成另一种语言。

等效性检查曾经找到过算术运算中的缺陷

等效性检查可以用于验证逻辑综合变换。它仅仅比较组合逻辑和时序逻辑，并不关心为满足严格的设计约束所进行的面向特定工艺的功能映射。数字设备公司（DEC，现在已合并到 HP 公司）的工程师们曾经用等效性检查发现过一个设计综合出错的结果。该设计使用了一个可综合的运算操作，这种运算在处理 48 位以上的数据时会发生错误。虽然在这个运算操作的文档里明确指出了这个缺陷，即它不保证可以处理 48 位以上的数据，但是综合工具并不知道这个文档指出的缺陷，因此产生了错误的逻辑。等效性检查发现了这个难以用门级仿真发现的问题。

1.5.3 模型检查

模型检查证实了关于设计行为的断言

模型检查是形式验证技术的一个新应用，它证明或否定了设计的断言或特征。例如，可以检查设计中的状态机是否有不可到达的状态或孤立状态。更强大的模型检查器可以判断状态机是否会发生死锁。

与接口相关的断言是另一类可以采用形式化验证的断言。关于设计接口的断言可以采用形式描述语言来描述，然后由工具来证明或否定这个断言。例如，一个断言可能表明：如果输出了 ALE 信号，那么 DTACK 信号或 ABORT 信号会随后输出。

最困难的是确定要证明哪些断言，并正确地描述这些断言

图 1.7 所示的是模型检查的恢复路径模型。模型检查技术的最大障碍是根据对设计的理解判断应该验证哪些断言。在这些断言中，只有一部分可以被证明。现在的技术还不能证明设计的高层次断言，即不能确定是否正确实现了复杂的功能。例如，如果能够证明以下断言会是很好的事：如果设定了一组特殊的寄存器，异步转移单元（ATM）将按某种次序产生一组输出。不幸的是，现在的模型检查技术还没有达到这个水平。

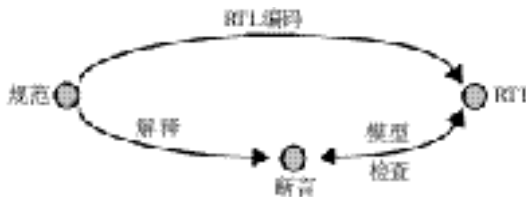


图 1.7 模型检查路径

1.5.4 功能验证

功能验证验证了设计意图

功能验证的主要目的是确定一个设计是否实现了预定的功能。图 1.8 所示为功能验证路径模型。功能验证检验了设计及其规范的一致性。如果没有功能验证，设计人员只能主观地认为自己没有错误地理解设计规范的意图，从设计规范的文档到 RTL 代码的变换是正确的。

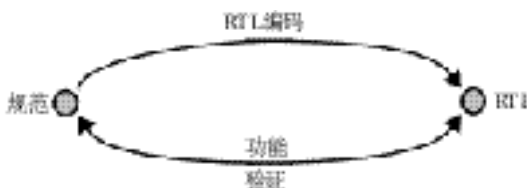


图 1.8 功能验证路径

可以证明缺陷存在，但无法证明缺陷不存在

除非设计规范用具有精确语义的形式描述语言编写^①，否则就不可能证明一个设计与设计规范相符，这一点非常重要。设计规范的文档是由设计人员用自然语言编写的，这些设计人员的表达与交流的能力各不相同。任何设计文档都需要理解。功能验证可以展示一个设计和设计规范相符，但不能证明这一点。我们只要找到一个不同点，就可以很容易地证明一个设计与预期功能不相符，但反之却不成立：没有人能证明没有不同点，就好像没有人能证明会飞的驯鹿和 UFO 不存在一样（但是只要找到一只会飞的驯鹿和 UFO，就可以证明相反的观点）。

1.6 功能验证的方法

功能验证可以通过以下三种方法实现：黑盒法、白盒法和灰盒法。

1.6.1 黑盒验证

黑盒验证不能检查或了解设计的内部

采用黑盒法进行功能验证时不用关心设计是如何实现的，所有的验证都可以通过接口完成，无需直接访问设计的内部状态，也不需要了解设计的内部结构和实现方法。这种方法显然缺乏对设计的可控性和可观测性，很难设置一种状态或隔离某种功能。黑盒法很难观测和输入相对应的输出，也难以对故障源定位，尤其是在故障的产生时刻和它在设计的输出端表现出来的时刻之间具有较长的时延时，更加难以定位。

^① 即使存在这种语言，设计人员也必须表明这种描述精确地描述了设计意图，而这只能通过某种更高层次的不确定的规范来解释。

测试用例和实现方式无关

黑盒法的优点是它不依赖于任何特定的实现方式，例如，单片 ASIC、RTL 代码、门、多片 FPGA、电路板或纯软件实现。黑盒功能验证方法是真正的一致性验证，不管具体的实现方式，它只检验设计是否实现了规范的要求。

我的母亲非常熟悉黑盒法：为了防止我们猜测圣诞礼物的内容，她从不在预先包装好的盒子上写上姓名^①。到了圣诞节，她不能打开它们，但又不得不分辨出每个盒子里装的是什​​么，然后才能知道这个盒子应该给谁。她曾经数次搞错了对象，成为节后我们的笑料。

黑盒验证很难控制和观测特定的功能

纯粹的黑盒法在现在的大规模设计中并不实用。一个数百万门的 ASIC 具有太多的内部信号和状态，从外部很难有效地验证所有的功能。深嵌在设计内部的关键功能很难控制和观测，而且设计的缺陷也不会轻易在 ASIC 的输出端口出现失效的征兆。例如，我们使用图 1.9 的黑盒 ASIC 级测试平台验证一个轮转式仲裁器。如果仲裁器在实现时对各个输入端没有平等对待，在输出端会有什么征兆？这种缺陷只有通过比较每个通道的实际吞吐量和理论吞吐量，经过多次长时间仿真的性能分析后才能发现。



图 1.9 底层功能的黑盒验证

1.6.2 白盒验证

白盒验证了解并控制设计的内部细节

从这种方法的命名就可以看出，白盒方法对设计的内部结构和实现具有完全的观测能力和控制能力。这种方法的优点是可以迅速设置感兴趣的状态和输入，或隔离一种特殊的功能，然后随着验证的进行观测输出结果，检查任何不同于预期行为的差异。

白盒验证与特定的实现相关连

然而，这种方法是与特定的实现紧密相连的。设计的修改意味着对测试平台的修改。在改变实现方式或重新设计后，这些测试平台将无法用于门级仿真。这种方法还需要了解设计的实现细节，这样才能设置要仿真的关键状态，确定要观测的结果。

^① 这让我的妻子很生气，因为她喜欢摇晃所有贴着她的名字的盒子。

可以使用白盒验证来增强黑盒验证

白盒验证是对黑盒验证的有力补充。它可以确保底层的特定功能实现的正确性，例如，计数器到达翻转值后是否翻转，数据路径是否已进行了适当的调整和排序。白盒方法可以只使用黑盒或灰盒的激励来检测功能的正确性。如图 1.10 所示，在图 1.9 的 ASIC 级黑盒验证环境中增加了白盒检查以验证仲裁器的功能。只要仲裁器的实现存在着不平等的情况，白盒检查就可以报告这个缺陷，这种错误报告方便了故障源的识别和定位。经过三个昼夜的仿真，通道吞吐量之间 2% 的差异可以很容易地解释为随机误差。



图 1.10 黑盒环境中的白盒检查

白盒方法用于系统级验证

纯白盒验证方法常用于 SoC 设计和系统级的验证。这里的系统由若干独立设计和验证过的模块构成。系统验证的目的是验证系统的功能，而不是再次验证各个模块的功能。由于系统的状态数量非常多，并且很难设置我们感兴趣的状态，因此系统级验证通常把系统看做是由很多黑盒构成的一个集合。每个模块看做一个黑盒，但整个系统是一个白盒，具有完全的可控性和可观测性。

1.6.3 灰盒验证

灰盒验证是就设计实现的相关性在黑盒验证和白盒验证之间的折中，前者不能控制设计的所有部分，后者不具备可移植性。

测试用例可能与其他实现方式无关

像黑盒验证方法一样，灰盒方法通过设计的顶层接口进行观测和控制。同时，针对特定实现的特殊功能进行专门的验证。对不同的设计实现可以进行相同的验证，但这种验证不会比其他黑盒验证更关注特殊功能。典型的灰盒测试用例可以增加测试覆盖率，输入激励被设计来执行特定的代码行或在设计中创建一系列的特定状态。如果设计结构（不是功能）发生了变化，这种测试用例仍然可以工作，但对提高测试覆盖率则不再有效。

增添设计功能以提高可观测性和可控性

典型的灰盒方法是对设计做某些非功能性的修改以提高可观测性和可控性，例如，增加可以通过软件访问的寄存器以控制或观测内部状态、加速实时计数器、强制引发异常或修改待处

理数据的数量以减少验证时间。这些增加的寄存器或功能在正常的操作中不会起任何作用，但在首个原型系统的集成阶段非常有用。

不能在设计阶段并行进行白盒验证

只有黑盒和灰盒方法才可以同时进行行为模型的功能验证和设计的实现。由于事先不知道任何设计实现的细节，因此只能选择这两种方法。

1.7 测试与验证的比较

测试验证了制造过程

测试很容易与验证相混淆。前者的目的是验证设计是否被正确地制造出来，后者的目的是确保设计实现了预期的功能。

图1.11所示的是测试与验证的恢复路径模型。测试检验了提交制造的网表和制造的芯片之间的一致性。

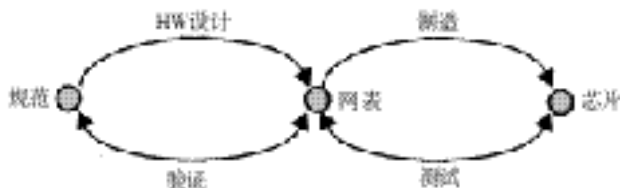


图 1.11 测试与验证的比较

测试验证了内部节点可以翻转

测试的过程是通过测试矢量进行的。这些测试矢量的目的并不是要执行某种功能，而是让设计内部的物理节点发生从0到1或者从1到0的变化，同时能从外部观测到这种变化。测试的物理节点数和总的物理节点数之比称为测试覆盖率。为了在测试矢量最少的情况下达到最大的测试覆盖率，测试矢量通常通过一个称为自动测试模式生成（automatic test pattern generation, ATPG）的过程自动产生。

测试的完整性取决于内部节点的可观测性和可控性

测试覆盖率取决于将内部节点设置成1或0，然后从外部观测这些节点是否真正被设置成1或0的能力。有些设计只有非常少的输入和输出，但却有大量的状态，因此需要很长时间才能控制或观测所有的内部节点。手表就是一个最好的例子：手表有3或4个输入端（位于表盘周围的按钮），少量的输出端（显示器上的数字和符号）。如果手表具有秒表和日历功能，它就有数十亿种可能的状态组合（把数百年分成毫秒计），需要几百年才能让这样一个设计遍历完所有可能的状态。

1.7.1 基于扫描的测试

将所有寄存器链接成移位寄存器可以增加可控性和可观测性

幸运的是,基于扫描的测试技术使这个问题变得易于处理了,它把设计中的所有寄存器连接成一个长的链。在正常模式下,寄存器正常工作,好像扫描链并不存在,如图1.12(a)所示。在扫描模式下,寄存器好像一个长的移位寄存器,如图1.12(b)所示。

要测试具有扫描链的设计,应把被测单元置于扫描模式下,然后通过扫描链把输入模式移入所有的内部寄存器。随后将被测单元设置成正常模式,并输入一个周期的时钟信号,将正常操作的结果送入寄存器中。最后把被测单元再次设置成扫描模式,并将结果从寄存器中移出(同时把下一组输入模式移入到内部寄存器),与预期的结果进行比较。

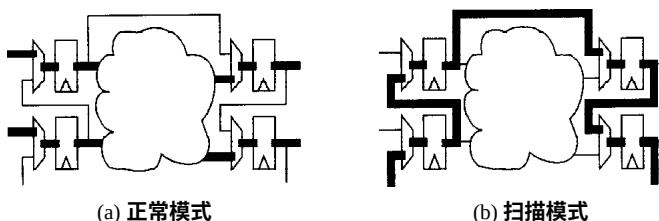


图 1.12 基于扫描的测试

基于扫描的测试对设计有一些限制

这种技术增加了设计的可控性和可观测性,也提高了测试覆盖率。为了能够插入扫描链并自动生成测试模式,必须对设计加以限制,这些限制包括但不限于:全同步设计、没有导出时钟或门控时钟、使用同一个时钟沿等。关于可测性设计的问题太大、也太复杂,不在本书讨论的范围之内。详情可参阅有关书籍^①与论文^②。

基于扫描的测试的优点远远超过了其限制的缺点

刚接触这种技术的硬件设计人员会反对这种限制,他们只看到眼前芯片面积的增加和他们所喜欢的设计方法被放弃。然而,当设计可以容纳一条或多条扫描链,当仅仅按一个键就可以实现测试模式和高测试覆盖率时,这些增加的面积和设计工作量就可以忽略不计了。对产品投放市场的信心的增强和所节省的时间带来的好处远远超过了基于扫描的设计带来的成本增加。

1.7.2 面向验证的测试

为满足可测性的要求,有可能需要修改设计,那么为满足验证的需求而修改设计是否合理呢?

① Abramovici, Breuer and Friedman, *Digital System Testing and Testable Design*. IEEE ISBN 0780310624.

② Cheung and Wang, "The Seven Deadly Sins of Scan-Based Design", *Integrated System Design*, Aug. 1997, p50-56.



在制定规范期间必须考虑验证

既然功能验证的工作量超过了电路设计的工作量,因此用额外的设计来简化验证是非常必要的。就好像在设计中加入扫描链增加了可测性、却没有增加设计的功能一样,为设计增加非功能性的结构来简化验证也应该是一个标准的设计过程。这种方法要求在项目的开始阶段,即规范的制定阶段就考虑验证的需求。在制定设计的结构时不仅要回答“要做什么”,还要回答“怎么验证”。

典型的面向验证的技术包括恰当定义的接口,在功能上明确区分相对独立的单元,提供额外的可以通过软件访问的寄存器以观测和控制内部节点,提供可编程多路复用器以隔离或旁路某个功能单元。

1.8 设计与验证的重用

今天,设计的重用已经成为了设计过程中的一部分,它是缩小在单个芯片上能制造出晶体管的数量和工程师能设计的晶体管的数量之间的差异的最好办法。这个差异也称为“产能差”。最初设计重用被认为是很容易实现的方法,但实践证明这是难以解决的。

1.8.1 重用涉及到信任

你不会使用你不信任的东西

设计重用最主要的障碍是文化方面的。设计人员一般不愿意把一个不了解的设计加入到自己的设计中。他们不相信别人的设计会像自己的一样好、一样可靠。设计重用的关键是获取他们的信任。

像质量一样,信任不是可以加入到设计中的某些东西。它必须是内在的,贯彻在整个设计过程中。它只能在使用可重用的设计的过程中获得:提供设计服务、与客户建立联系。只有建立了信任关系,设计重用才能继续下去。

恰当的功能验证论证了设计的可信度

可信度可以通过恰当的验证过程来证明。通过向用户展示已经详细深入地按照设计规范进行了验证,可以迅速取得用户的信任。功能验证是证明设计已经满足、甚至超过了工程师能完成的类似设计的品质的惟一方法。

1.8.2 面向重用的验证

可重用设计的验证必须满足更高的置信度

在创造一个新的设计时,设计人员会对自己的能力充满信心,也会信任所做设计的正确性。功能验证用来验证这个观点,以及验证设计的薄弱之处。如果要重用一個设计,只能依靠

功能仿真来建立这种自信和信任。因此，可重用设计必须经过比专用设计更严格的验证，以满足更高的置信度。

所有的功能、配置和使用情况都需要验证

为了满足各种环境和应用的需要，可重用的设计往往是可配置或可编程的，因此必须验证各种可能的配置和使用环境。设计的所有功能都需要验证并演示给用户。

1.8.3 验证重用

测试平台中的模块也可以重用

正如设计可以重用一样，测试平台的部分模块也可以重用。例如，图 1.13(a)中用来验证设计模块的总线功能模型（BFM）也可以用来验证图 1.13(b)中使用该模块的系统。

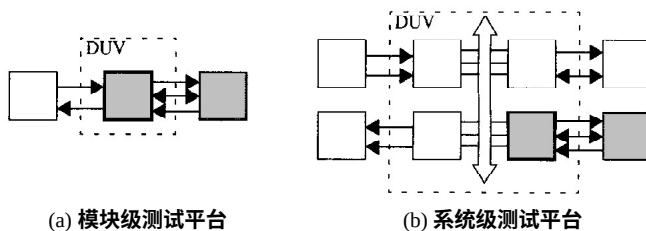


图 1.13 在系统级测试平台中重用模块级测试平台的 BFM

验证重用有自己的需要

有不同程度的验证重用，有些很容易实现，有些则面临着与设计重用相同的问题。在不同的测试平台和测试用例之间为相同的设计重用 BFM 是非常简单的，只需构造一个合适的验证环境。在同一个设计的不同版本之间重用测试平台的模块和测试用例存在一些困难，例如，无法验证新版本的新功能。在不同的项目之间或不同的抽象层次之间重用设计面临更多的问题，包括验证模块本身的设计等。

复用并不是重用

复用 (Salvage) 指重复使用已有测试平台的一部分，这一部分测试平台并不是专门为重用而设计的。复用模块的可用性在很大程度上取决于要验证的设计和原始设计的相似程度。例如，用来验证接口模块的 BFM 可能就不适合验证一个使用接口模块的系统，如图 1.13(a)所示。

模块级和系统级测试平台对 BFM 的需求并不相同

模块级验证必须检验实现接口协议的状态机和译码器，这种验证需要使用事务级 BFM，并能够控制协议信号，改变时序或注入协议错误。系统级验证要检查接口模块后面的高层功能，它要求能够将高层数据封装到接口事务中，它所要求的可控性位于比验证接口模块的信号更高的层次。

1.9 验证的成本

验证是不可避免的，它总是需要耗费很长的时间、人力和物力。验证不能直接产生任何收益，最后销售并获利的是经过验证的设计，而不是验证本身。然而，验证是不可缺少的，设计必须保证功能正确并能为客户带来利润才能最终投入市场并获益。

随着错误的减少，寻找错误的时间越来越长，成本越来越高

实际上，验证是一个永远也不会结束的过程。验证的目标是确保一个设计没有错误，但这个结论是无法证明的。验证只能检测出错误的存在，而不是证明并不存在错误。如果有足够多的时间，错误总是能被找到。这样问题就出现了：是否值得花费很大的代价去寻找这些错误？随着用在验证上的时间越来越多，相同的开销所能找到的错误越来越少。随着验证过程的深入，所得到的回报越来越少，找到剩余的错误的代价也越来越高。

功能验证和统计学上的假设检验类似。需要验证的假设是“我的设计功能是否合格？”答案可以是“合格”或“不合格”。但任一种答案都有可能是错的，这些错误的答案可以分为第一类错误（伪错，false negative）或第二类错误（漏报错误，false positive）。

必须避免“漏报错误”

图 1.14 是可能发生的各种错误情况的示意图。第一类错误，即“伪错”，是比较容易发现的一种错误。这种情况表示验证过程发现了一个实际上并不存在的错误。发现这种错误后，通过修改验证过程，将判决结果从“不合格”改成“合格”就可以避免这种错误。第二类错误是严重的错误：验证过程漏掉了一个错误。在第二类错误，即“漏报错误”的情况下，错误的设计不知不觉地通过了检验，这将会带来潜在的风险。

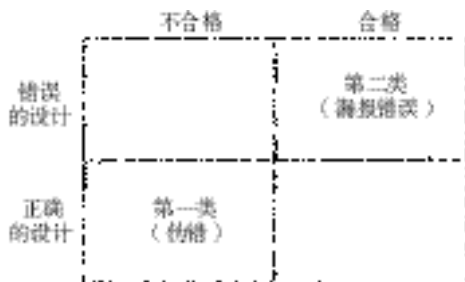


图 1.14 第一类错误和第二类错误

在确定存在潜在的、灾难性的后果方面，美国食品及药物管理局面临着第二类错误的风险。例如，临床实验结果正确是否意味着一种危险的药剂可以投放市场？错误的设计被放行的后果可能导致产品被召回，也可能导致空间探测器着陆后完全失效。



随着公司的未来可能面临的潜在风险，验证过程中一个价值为64 000美元的问题是：“验证到什么程度才足够？”本书所讲述的功能验证过程和下一章介绍的一些验证工具将回答这个问题。

验证过程中一个价值为六千四百万美元的问题是：“什么时候才能完成验证？”尽管不能精确地估计时间，但确定正在进行验证过程的哪一步比估计验证所需的时间要容易得多。第3章介绍的验证规划过程使验证主管可以更好地估计完成任务的时间和效果。

1.10 小结

验证是一个过程，而不是一系列测试平台的集合。

验证独立于设计规范及其实现方式。要理解这种独立性，必须明白到底要验证什么？

验证可以在不同的设计层次进行，不同的层次设计其可测程度也不相同。笔者更喜欢使用“黑盒法”，因为用这种方法可以写出可移植的测试平台。应该根据需求来选择灰盒测试平台或白盒测试平台。

在设计开始阶段就应该考虑验证的问题。如果一个功能难以验证，那就应该修改设计，增加这个功能的可测性和可控性。

第2章 验证工具

正如上一章所提到的，自动化是用来提高效率 and 可靠性的一种方法。本章介绍功能验证环境中使用的工具。这些工具，例如仿真器，对于功能验证起着至关重要的作用。其余的工具，例如 Lint 或代码覆盖工具，使一些单调的验证工作自动化，并增加了功能验证结果的可信度。

本章没有提到所有的工具，也没必要使用这里提到的所有工具

没有必要使用本章涉及的所有工具，随着新的专用和通用自动化验证工具不断地涌入市场，我们所能列举的工具是有限的。作为一名验证工程师，你的工作是使用必要的工具来确保验证过程的结果没有第二类错误，即漏报错误。作为项目主管，需要在预算之内按期交货，你的职责是为工程师提供合适的工具，使他们高效可靠地完成工作。最重要的是，你还需要判断寻找功能错误的成本何时将超过所带来的效益。这些工具能提供信息，帮助你判断何时能达到这个目标。

本书不倾向使用特定的商业化工具

笔者在这里提及了一些商业工具的名称，仅仅是因为需要，并非是因为个人的倾向性。这里，向未提及的同类产品的供应商道歉，未提及的原因并非是因为产品等级的差别而是由于笔者知识有限。此外，所提及的所有商标及服务标识，无论注册与否，其所有权都归属各自的厂商。

2.1 lint 工具

lint 工具可发现程序员易犯的一般性错误

术语“lint”来自于一个 UNIX 工具的名字，该工具可以解析 C 程序并报告错误的用法和潜在的问题。当 C 程序设计语言被 Dennis Ritchie 开发出来时，它并不包含一些后续版本（例如 ANSI-C、C++）或其他强类型语言（例如 Pascal、ADA）使用的检错结构。lint 发展成为一种用来识别程序员的一般性错误的工具，使人们更快、更有效地发现错误，而不是等到程序执行出错时再去发现问题。

lint 能识别出真实的问题，例如，自变量和函数调用之间的类型不匹配，或自变量数量的不匹配，如例 2.1 所示。这段源代码在语法结构上是正确的，并且在 gcc 版本 2.8.1 中也不产生任何编译错误或警告。

例2.1 语法结构正确的K&R C源代码

```
int my_func(addr_ptr, ratio)
    int *addr_ptr;
    float ratio;
{
    return (*addr_ptr)++;
}

main()
{
    int my_addr;
    my_func(my_addr);
}
```

但是，例 2.1 出现了两个不合理的严重错误：

1. my_func 函数调用中只有一个参数，而不是两个。
2. my_func 函数调用中的第一个参数是整数值，而不是整数指针。

能够比执行程序更快地发现问题

如例2.2所示，lint能识别出这些问题，使程序员在执行程序前及时更正以避免程序运行失败。在执行程序时诊断这些问题需要花上几分钟的时间运行调试器，而使用lint只需要几秒钟，显而易见，后者效率更高。

例2.2 例2.1的lint 输出

```
src.c(3): warning: argument ratio unused in
function my_func
src.c(11): warning: addr may be used before set
src.c(12): warning: main() returns random value
to invocation environment
my_func: variable # of args.    src.c(4)  ::
src.c(11)
my_func, arg. 1 used inconsistently
src.c(4)  :: src.c(11)
my_func returns value which is always ignored
```

lint 工具是静态的

lint 工具和其他验证工具相比有极大的优点：不需要激励，也不需要描述期望的输出，完全静态的检验、lint 工具内嵌了期望值。

2.1.1 lint 工具的局限性

lint 工具只能识别出某些类型的问题

lint 也能识别出其他潜在的问题。在修改了例 2.3 中的所有错误后，lint 仍然报告函数 my_func 的调用存在问题：返回值总是被忽略。lint 工具不能识别出所有源代码中的问题：它只能通过静态推理代码结构来发现问题，不能发现算法和数据流中的问题。

例如，在例2.3中，lint 没有意识到未初始化的 `my_addr` 变量将在函数 `my_func` 中递增而产生随机结果。lint 工具类似于拼写检查程序：能识别拼错的单词，但不能识别是否用错了单词。例如，本书中可能有多处将“width”误用为“with”，这样的错误便不能被拼写检查程序（或 lint 工具）发现。

例2.3 功能正确的K&R C源代码

```
int my_func(addr_ptr)
{
    int *addr_ptr;
    return (*addr_ptr)++;
}

main()
{
    int my_addr;
    my_func(&my_addr);
    return 0;
}
```

许多伪错出现在报告中

lint 工具的局限性还在于经常报告伪错。为了避免第二类错误——漏报错误，lint 工具经常过于谨慎地报告出实际不存在的错误，从而导致许多第一类错误——伪错。报告中的此类问题常使设计者浪费时间和精力，也因此导致一些人放弃使用 lint 工具。

仔细过滤错误信息

使用 lint 工具时应该过滤其输出的伪警告和伪错误，这可以减少寻找实际不存在的问题的几率。更重要的是，这降低了输出的混乱程度，即伪错以外的真正错误被忽略的可能性降低了。同理，真正的错误应该被强调，所以在写这样的过滤器时应非常谨慎以保证真正的错误不被过滤掉。

命名规则可以帮助输出过滤

恰当定义的命名规则可以有效地确认某个警告是否重要。例如，在例 2.4 中，其名称以“_lat”结尾的信号推论出锁存器的报告是伪警告，其他的报告可视为真正的错误。

例2.4 一段假设的Verilog lint 工具的输出

```
Warning: file decoder.v, line 23: Latch
inferred on reg "address_lat".
Warning: file decoder.v, line 36: Latch
inferred on reg "next_state".
```

不要关闭检查功能

过滤 lint 工具的输出比在源代码或命令行里关闭检查功能更有优势。因为检查可能在需要检查的时间段内保持关闭，导致忽略真正的问题。当新的源文件加入到项目中时，一些看似不相关的检查有可能变得非常重要。

在编写代码时就使用lint

显然，最好的办法是在问题刚产生时就解决它，所以应该在写源代码的同时就运行 lint。如果等到大量代码编写完成后再进行 lint，那么夹杂着伪错的大量报告将难以处理。因此当程序员对编写的代码仍保持着清晰的思路时，是判断报告真伪的最好时机。

强调编码规则

通过使用用户定义的规则，lint过程强化了编码规则和命名规则^①。所以为了保证代码能符合可读性和持续性的要求，lint 应该成为整个过程中不可缺少的一部分。

2.1.2 Lint Verilog 源代码

Lint Verilog 源代码能检查出一般性错误

Lint Verilog 源代码能保证所有数据都经过了适当的处理，没有意外的删减或增加。例2.5 中所示的 Verilog 模型看似正确，编译也没有出错，但在 Verilog-95 的一些情况下会产生意外的结果。

例2.5 存在潜在问题的Verilog 代码

```
module tristate_buffer(in, out, enable);  
parameter WIDTH = 8;  
input  [WIDTH-1:0] in;  
output [WIDTH-1:0] out;  
input          enable;  
  
assign out = (enable) ? in : 'bz;  
endmodule
```

在大多数情况下，错误可能并不明显

上例中的错误在于：赋值过程的输出信号“out”和常数“bz”之间的位宽不匹配，未约束常数的位宽为 32 位，但输出的宽度根据使用对象的不同而不同。只要输出数据的宽度小于或等于 32，程序将正确运行：常数的值将被截断到与输出位宽相匹配。

但是，在 Verilog-95 中，输出宽度大于 32 位时会发生错误：Verilog-95 在常数中补零以与输出的宽度相匹配，因而导致发生错误。低 32 位被置为高阻态，而所有其他高位被置为 0。Verilog-2001 已经修改了这种错误。

一般仿真不能发现此类错误，除非使用了大于 32 位的配置，并且产生的错误结果恰好被观察到，然而 lint 工具总能在几秒之内发现这类错误。

2.1.3 Lint VHDL 源代码

由于 VHDL 是一种强类型的语言，所以它不需要像 Verilog 那么多的 lint。VHDL 编译器执行了许多本应由 lint 工具执行的检查，尽管如此，最好还是使用 lint 工具来识别潜在的错误。

^① 见附录 A：编码规则。

lint 能发现额外的多驱动器

例如，使用STD_LOGIC 类型容易引起一种常见的错误。由于STD_LOGIC 是一种判决类型，所以STD_LOGIC 信号可能有一个以上的驱动器。在为硬件建模时，多驱动器信号只能有一种情况：总线。在所有其他情况下（99%以上），一个信号只能有一个驱动器。从例2.6所示的程序中可以看出，一个简单拼写错误很容易被VHDL 编译器忽略。

例2.6 错误的多驱动器

```
library ieee;
use ieee.std_logic_1164.all;
entity my_entity is
    port (my_input: in std_logic);
end my_entity;

architecture sample of my_entity is
    signal s1: std_logic;
    signal s1: std_logic;
begin
    statement1: s1 <= my_input;
    statement2: s1 <= not my_input;
end sample;
```

拼写错误能导致严重的问题

在例2.6中，标注为statement1 和 statement2 的并发信号赋值语句同时对信号“s1”(S-1)赋值，而信号“s1”(S-L)仍未被赋值。假如用STD_ULOGIC 类型代替STD_LOGIC 类型，则VHDL 工具会在发现与非判决信号相连的多驱动器后报告错误。但是，不能保证STD_ULOGIC 类型用于所有单驱动器的信号。如例2.7所示，无论是哪种类型，都需要使用Lint 工具报告多驱动信号。

例2.7 一个假定的VHDL lint 工具的输出

```
Warning: file my_entity.vhd: Signal "s1" is
multiply driven.
Warning: file my_entity.vhd: Signal "s1" has no
drivers.
```

使用命名规则来过滤输出

识别总线信号和忽略相关警告的工作应该由程序员来完成。对于此类信号使用命名规则，会帮助程序员更好地识别这些可以被忽略而不影响程序正确性的警告，还可以提高代码的可靠性。例2.8说明了一个命名规则的例子，可以用“_bus”后缀来命名任何总线信号^①。

^① 关于命名规则的例子，参见附录A。

例2.8 多驱动器信号的命名规则

```
--  
-- data_bus, addr_bus and sys_err_bus  
-- are intended to be multiply driven  
--  
signal data_bus : std_logic_vector(15 downto 0);  
signal addr_bus : std_logic_vector( 7 downto 0);  
signal ltch_addr: std_logic_vector( 7 downto 0);  
signal sys_err_bus: std_logic;  
signal bus_grant : std_logic;
```

lint 能识别暗含的锁存器

lint 工具不仅能识别多驱动器的问题，也能识别出可综合代码中不希望的锁存器，还可以识别编码规则。

2.1.4 Lint OpenVera 和 e 源代码

因为 OpenVera 和 e 都是强类型语言，所以它们不需要像 Verilog 那样多的 lint。但和 Verilog 相似，最好使用 lint 工具来检查潜在的问题。例 2.9 说明了两个并发执行语句的竞争情况导致不确定的结果（这种竞争情况的具体解释见 4.8 节）。lint 工具可以轻易地查出这种类型的错误。用于 OpenVera 和 e 的 lint 工具已经开始出现。

例2.9 OpenVera 源代码的竞争情况

```
{  
    integer i;  
    fork  
        i = 1;  
        i = 0;  
    join  
}
```

2.1.5 代码检查

代码检查一般在同事之间进行

虽然代码检查在技术上与 lint 工具不同，但目标是相同的：在功能验证和仿真之前寻找功能和代码风格的错误。lint 工具只能识别是否使用了有问题的语言，不能判断代码是否真实表达了预期的行为。在代码检查中，一个设计者编写的源代码可以让几位程序员共同提出意见，这样做是为了检查出那些自动化工具不能识别的问题。代码检查能够找出设计意图和实现之间的差异，同时也为改进代码，例如更好的注释、结构或组织提供了机会。

代码检查能识别定性的问题和功能错误

进行代码检查是评价源文件的可维护性以及注释是否合适的最好做法，也可以识别代码风格等定性的问题。如果代码很容易理解，那么也就很容易发现其中的功能错误和遗漏问题。

进行代码检查并不是新观点,在软件设计业中已经被使用多年。关于怎样进行有效的代码检查的具体信息,见 <http://janick.bergeron.com/wtb> 的 resource 部分。

2.2 仿真器

在实现设计以前应进行仿真

仿真是最普遍和常见的验证工具。之所以命名为仿真器,是因为它的作用是使设计趋近于实际情况。仿真并不是一个项目的最终目的,所有硬件设计项目都是为了创造可以销售并盈利的真实物质。仿真器试图创造一种人工的环境来模拟真实的设计,这种验证使设计者能在产品制造之前调整设计,改正错误和问题。

仿真器只是实际情况的近似

我们必须清楚地明白仿真器只是实际情况的近似。为了仿真能顺利进行,许多物理性质被简化甚至忽略了。例如,四状态的数字仿真器就假定信号的可能值只有0、1、未知状态、高阻状态四种,而在实际的物理和模拟信号情况下,信号值是流过铝导线或铜导线的电压和电流量的连续函数,具有无数个可能的值。在离散的仿真器中,在5 ns后发生的若干个确定性事件在物理世界里可能并不同时发生,而是随机发生。

仿真器受仿真对象的支配

在简化的仿真环境下,仿真器可以做的惟一工作就是执行用某种语言描述的设计,这种描述使用明确定义的语言,有准确的语义。描述必须准确地反映试图模拟的实际情况,否则设计者将无法知道正在仿真的设计与最终制造的产品之间的差别。由于不能证明没有错误,功能的正确性和模型的准确性就是一个大的问题。

2.2.1 激励和响应

仿真需要激励

仿真器不是静态工具。静态的验证工具在工作时不需要任何用户给出信息或做任何操作,例如 lint 工具就是静态工具。但仿真器却不同,它需要模拟设计工作时的真实环境,即测试平台(testbench)。本书的主要目标就是编写测试平台,它为设计提供输入信号,使仿真器能基于该输入信号模拟设计的响应。

仿真的输出是在仿真器外根据设计意图进行比较验证的

另外必须牢记的是,仿真器并不知晓设计者的意图,因此它不能判断被仿真的设计是否正确。设计的正确性应该由验证工程师根据仿真的结果做出相应的判断。一旦设计在模拟的环境下进行仿真后,验证工程师的主要职责就是检查相应的输出,判断响应是否符合设计的意图。

2.2.2 事件驱动的仿真

仿真器的速度永远不能满足需要

仿真器一直面临着一个难以克服的问题：它的速度十分有限。在仿真器试图模拟的现实物理世界中，电流以光速传播，数百万个晶体管每秒翻转上亿次。即使在理想情况下，仿真器使用的通用型计算机每秒最多也只能处理十亿条指令。现实物理世界比仿真器永远有着速度上的优势。

只有当输入信号变化时输出才会变化

优化仿真器性能的方法之一是避免仿真那些不需要的情况。以图 2.1 中的二输入异或门为例，如果输入信号不发生变化，如图 2.1(a) 所示，只要电源电压保持不变，电流将持续地流过晶体管（在某些半导体工艺中），半导体中的微观粒子将持续的移动，呈现的二值输出状态（逻辑 1 或逻辑 0）也将保持不变。只有当某一个输入信号改变时，如图 2.1(b) 所示，输出才会改变。

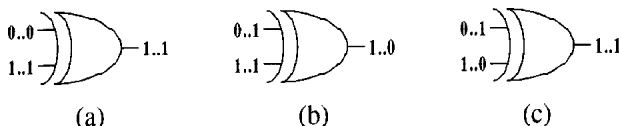


图 2.1 异或门行为

称为事件的信号值的改变将驱动仿真器进行仿真

例 2.10 是一段用 VHDL 描述的或非门代码。仿真器可以连续地运行该代码，如果输入值不变的话，将得到相同的输出值。显而易见，当输入不变时，不运行代码可以提高仿真器的性能。换种说法就是：仅仅在输入改变时才运行代码，因此输入信号的改变驱动了仿真器的仿真过程。如果我们定义一个输入信号的改变为一个事件 (event)，那么这种仿真器就是事件驱动 (event-driven) 的仿真器。

例 2.10 或非门的 VHDL 程序

```
XOR_GATE: process (A, B)
begin
    if A = B then
        Z <= '0';
    else
        Z <= '1'
    end if;
end process XOR_GATE;
```

输入信号的改变不一定会引起输出的改变

当图 2.1(c) 中的两个输入信号都改变时会发生什么情况呢？按逻辑推断，输出应保持不变。那么此时事件驱动的仿真器是否运行代码呢？出于以下两个原因，仿真器应运行这段或非门代码。首先，在真实世界或非门的输出确实会发生变化，输出有可能在几百 ps 内在 0 和 1

之间振荡或处于既非0也非1的状态（见图2.2），取决于模型的准确性。可以选择包括了短暂停留在不确定状态（x）的或非门模型，这样就可以更准确地仿真两个输入信号同时改变的情况。

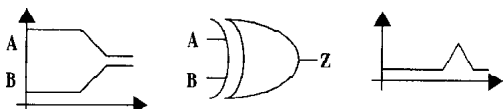


图 2.2 两个输入信号都改变时的异或门行为

描述输入和输出之间关系的代码是任意的

其次，事件驱动的仿真器并不知道要仿真一个异或门，它只知道在运行一段二输入单输出的代码。图2.3是从仿真器角度所看到的异或门，它是一个简单的二输入单输出的黑盒。黑盒里可能包含一个二输入的与门（在这种情况下，如果两个输入信号都改变，输出也可能改变），也可能是一个1024位的线性反馈移位寄存器（LFSR）。



图 2.3 或非门的事件驱动的仿真器视图

事件驱动的仿真机制具有某些局限性和有趣的副作用，将在第4章中进一步讨论。

事件驱动的仿真器一般都有加速选项

仿真器的供应商及其竞争对手不断推出更快、更易使用的仿真工具。通过简化设计或者仿真算法中的一些潜在假设，可以提升事件驱动的仿真器的性能。例如，将延时降低为相同的单位延时或者使用两状态（0和1）来代替四状态（0，1，x和z）都可以加速仿真。可以查阅仿真器的文档来确定有哪些加速选项，以及了解使用这些加速选项所带来的准确性降低方面的后果。

2.2.3 基于周期的仿真

图2.4是从事件驱动角度看到的一个同步电路，它在两个边沿触发的触发器之间有三个二输入的与门，这些器件构成一个链。假设Q1保持0，Q2保持1，其他输入保持不变，时钟输入端的时钟上升沿将使事件驱动的仿真器按如下所示的方式仿真该电路：

1. 时钟输入端的事件（上升沿）使仿真器运行描述触发器的代码，在1 ns的延时后Q1的输出变为1，Q2的输出变为0。
2. Q1上的事件使仿真器运行描述与门的代码，在2 ns的延时后S1的输出变为1。

3. S1 上的事件使仿真器运行描述或门的代码，在 1.5 ns 的延时后 S2 的输出变为 1。
4. S2 上的事件使仿真器运行描述异或门的代码，在 3 ns 的延时后 S3 的输出变为 1。
5. 下一个时钟上升沿使仿真器运行描述触发器的代码，在 1 ns 的延时后 Q1 保持不变，Q2 变回 1。

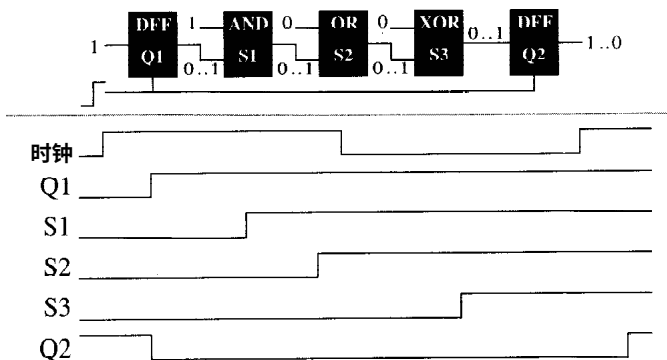


图 2.4 同步电路的事件驱动的仿真器视图

同步电路中的许多中间事件并不影响功能

要仿真这个简单电路的一个时钟周期的行为，需要产生六个事件并运行七个模块（有些模块可能运行两次）。如果我们感兴趣的仅仅是 Q1 和 Q2 的最终状态，而不是中间的组合逻辑信号，那就可以通过只仿真 Q1、Q2 的重要事件来优化（时钟的有效沿）。也就是说，仿真是基于时钟周期的，这就是基于周期的仿真器的工作原理。

图 2.4 中的同步电路可按如下的顺序在基于周期的仿真器中模拟：

基于周期的仿真器将组合逻辑转化为表达式

1. 描述电路的代码被编译以后，所有组合逻辑转化为一个表达式，用来产生基于扇入触发器当前状态的所有触发器的输入值。

例如，下面是 Q1、Q2 之间组合逻辑的初始描述：

```
S1 = Q1 & '1'
S2 = S1 | '0'
S3 = S2 ^ '0'
```

编译后成为这个表达式：

```
S3 = Q1
```

图 2.5 所示是编译后产生的电路的基于周期的仿真视图。

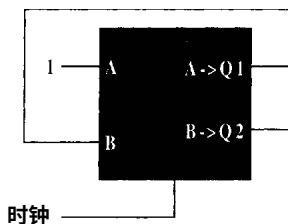


图 2.5 同步电路的基于周期的仿真器视图

2. 在仿真过程中,无论时钟的上升沿何时到来,预编译的组合逻辑函数的返回值将作为所有触发器的输入值更新所有的触发器。

使用基于周期的仿真器仿真上述电路,需要产生两个事件并运行一个模块。无论输入“ A ”是否改变,这两个事件运行的逻辑计算量都是相同的。只要执行逻辑计算的时间比调度中间事件的时间少^①,而且在每一个时钟周期内都有许多寄存器的状态改变,基于周期的仿真就会有更大的优势。

基于周期的仿真不带有时序信息

仿真性能的提高是有代价的:仿真中所有的时序和延迟信息都被省略了。基于周期的仿真器假定整个设计满足所有触发器的建立和保持时间的要求。使用基于周期的仿真器仿真时,时序通常由静态时序分析器验证。

基于周期的仿真器只能处理同步电路

基于周期的仿真器进一步假设有效的时钟边沿是转换状态的惟一重要事件,其他所有输入都被假定为与该有效时钟沿同步,因此它只能仿真全同步设计。任何包括非同步输入、锁存器或多时钟域的设计都无法准确仿真。幸运的是,静态时序分析适用同样的限制,所以适合用基于周期的仿真来验证功能的电路也适合用静态时序分析器来验证时序。

2.2.4 联合仿真

实时上没有哪个设计和测试平台完全适合于某个仿真器、仿真算法或建模语言。设计不同元件可能使用不同的语言描述;设计也可能包含不能被基于周期的算法仿真的部件。设计应该用 VHDL 或者 Verilog 来编写,而测试平台应该用 HVL 来编写。

多个仿真器能处理一个仿真中的不同部分

为了处理设计中不满足基于周期的仿真的要求的部分,大部分基于周期的仿真器都集成了事件驱动的仿真器。如图 2.6 所示,设计的同步部分由基于周期的算法仿真,而余下的由常规

^① 大多数情况下确实是这样。

的事件驱动仿真器仿真。两个仿真器（事件驱动的和基于周期的）同时运行，联合对整个设计仿真。

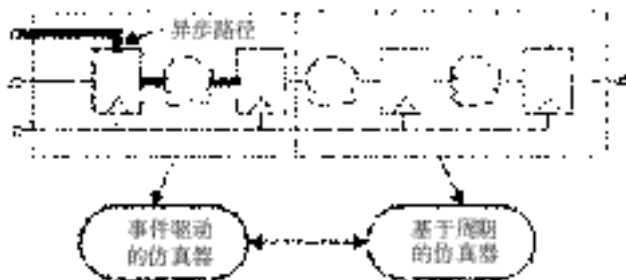


图 2.6 事件驱动和基于周期的联合仿真

很多广泛使用的联合仿真环境都能进行VHDL和Verilog、HDL和HVL或者数字和模拟联合仿真。例如，图2.7就是用一个测试平台（用e编写）和设计，采用Specman Elite和HDL仿真器联合仿真。

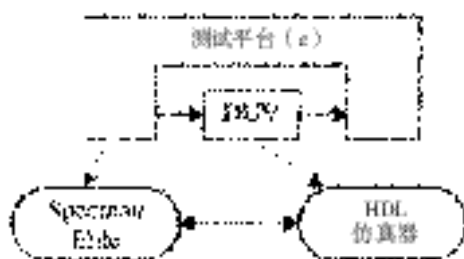


图 2.7 HVL 和 HDL 联合仿真

所有的仿真器按锁定的步骤工作

在联合仿真过程中，所有的仿真器围绕时间轴按锁定的步骤运行，即同时处于仿真时刻T1并且同时到达下一个时刻T2。这意味着联合仿真环境的速度取决于最慢的仿真器。在一些还处于试验阶段的联合仿真环境实现了时间封装同步，即允许一些仿真器超前于其他仿真器。

通信和同步开销降低了联合仿真的性能

联合仿真的最大障碍来自于仿真器之间的通信开销。当一个仿真器产生的信号是另一个仿真器的输入时，该信号的当前值和值改变的定时信息必须被传递。这种通信通常涉及一个仿真器中的事件到另一个仿真器中对等事件的转换。当两个仿真有着不同的语义时，转换中就会发生混乱。语义的不同经常出现，因此解决语义差别成为联合仿真的先决条件。

在仿真器间转换值和事件会导致混乱

转换混乱的例子非常多。例如，Verilog 的 128 个可能状态（由正交的逻辑值和强度组成）如何映射到 VHDL 的 9 个逻辑值？如何将模拟仿真器中的电压和电流值转换成数字仿真器中的逻辑值和强度？如何将 x 或 z 值转换成二状态的 e 值？如何将 Verilog 中的零延时事件（Verilog 没有严格的 Delta 周期概念）^①转换成 VHDL？

不要混淆联合仿真与单核仿真

如图 2.8 所示，联合仿真是两个以上的仿真器共同仿真一个设计，每个仿真器只仿真设计的一部分。和联合仿真不同的是，有些仿真器可以编译采用不同语言混合描述的设计模型代码，例如，Cadence 的 NCSIM 仿真器和 Model Technology 的 ModelSim 仿真器都可以仿真用 VHDL 和 Verilog 语言混合描述的设计，Synopsys 的 VCS 仿真器可以仿真 Verilog 语言和 OpenVera 语言的子集。如图 2.9 所示，所有的语言被编译成一个内部的表达式或机器码，仿真由单个的仿真内核运行的。



图 2.8 联合仿真



图 2.9 混合语言的仿真器

2.3 知识产权的验证

可以购买 IP 核实现标准功能

要验证一个设计，必须有所有单元模型才能仿真。RTL 设计的模型是设计工作的副产品，也是仿真的实际目标。验证工作还需要嵌入式或外部 RAM、标准接口和现成的模型。如果能从第三方供应商手中获得 RAM、IP 核、设计规范或标准部件，则也能从他们那里得到相应的模型。有时器件供应商并不提供模型，那就不得不从其他的供应商处获取模型。

① 有关 Delta 周期的详细内容，参见 4.7.5 节。

购买模型比自己编写花费更少

从第三方供应商手中买一个仿真模型乍看来非常昂贵,于是很多人就选择编写自己的模块来节约这笔购买专利的花费。但是,你应该判断一下这种付出在经济上是否真的富有成效:你是从事建模还是从事芯片设计?如果缺少合格的工程师,为什么将有限的资源用在编写一个不能体现公司任何竞争优势的模型上?如果自己设计并不值得,那编写自己的模型还有什么意义呢?

自己编写的模型不如购买的模型可靠

其次,你编写的模型以前从未使用过,其质量要比那些被其他公司使用过的模型差得多。对于一个功能正确且可靠的模型,其价值远远高过一个不确定的模型。通常编写并验证一个模型以达到跟购买的模型同样的可靠程度的费用比购买模型更昂贵。可以肯定的是,无论模型多么简单(例如一个四组的二输入与非门,74LS00),第一次设计时都容易出错,不是功能上的错误,就是时序或者连接性方面的问题。

验证IP核的供应商有很多,许多是用HVL或C代码编写的;其他的用不可综合的VHDL和Verilog代码。出于保护IP和授权方面的考虑,大部分验证IP核都以编译过的二进制模型方式提供。验证IP核包括外部和嵌入式存储器的功能模型、标准接口的总线功能模型、协议发生器和分析器、标准协议的断言集、可用器件和处理器的黑箱模型等。

2.3.1 硬件模拟器

如果找不到可以购买的模型怎么办

可能会碰到这样的情况:所需的设计模型太新或太复杂,以至没有供应商能及时设计出可靠的模型。例如编写本书的第一版时,至少有两家供应商提供Pentium处理器的全功能模型,但没有供应商提供Pentium III的模型。如果你的PC板里用到了最新的Intel微处理器,想在制造它之前验证其功能是否正确,就必须用其他的办法来加入这个微处理器的仿真模型。

可以将芯片插入仿真器

硬件模拟器可以解决这个问题,它是一个连接到仿真器的小盒子。硬件模拟器上装载了需要被仿真的真实的、物理的芯片,在仿真过程中与仿真器通信(通过特殊的封装接口),从仿真器获取输入信号并输入给器件,然后将输出的采样值从器件传回到仿真器。图2.10说明了这种通信过程。



图 2.10 通过接口连接的硬件模拟器和仿真器

仍然需要建模仿真I/O信号的时序

使用硬件模拟器并非一项简单的工作,通常还需要设计一个转接板,使器件和模拟器的插座相适应。另外,模拟器不能进行器件的输入时序检查,也不能准确反映输出的延时。因此,为了更准确地模拟一个器件,必须编写执行这些时序检查和延时的代码。

硬件模拟器具有更好的仿真性能

硬件模拟器也适合仿真有一定抽象水平要求的模型。一个可以取指、译码并执行指令的全功能的现代处理器模型,在可接受的有限时间内只能执行10到50个指令,而真实的物理器件只需要几毫秒。因此使用硬件模拟器可以极大地提高电路板级和系统级的仿真性能。

2.4 波形浏览器

波形浏览器显示出随时间变化的信号值

波形浏览器通常和仿真器一起,它能形象地呈现出各种信号随时间的变化,以及各个信号变化之间的关系。用它可以特殊的时间段上缩放,测量两个变化之间的时间差,或者以位串、二进制、十六进制或符号方式显示多位宽度的信号。图2.11是一个显示了四位同步计数器的输入和输出的波形浏览器视图。

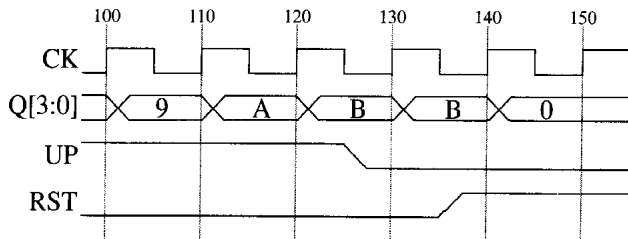


图 2.11 四位同步计数器的波形显示

使用波形浏览器来调试仿真

波形浏览器在修改设计或测试平台的阶段是必不可少的。波形浏览器可以方便地检查代码的行为是否和预期的一样,可以有效地诊断设计或测试平台的问题出在何处以及为什么会出现问题。波形浏览器可以在仿真过程中交互地使用,更重要的是它能在仿真结束后脱机使用。如图2.12所示,波形浏览器能回放记录在波形数据文件中的仿真事件。

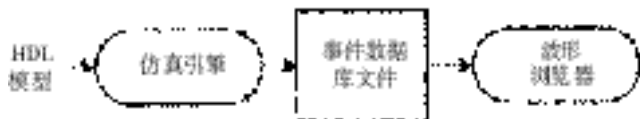


图 2.12 波形浏览作为仿真的后处理

记录波形数据降低了仿真的性能

浏览波形作为后处理的一步使你能快速浏览一个运行了几小时的仿真。然而，记录波形数据信息极大地降低了仿真器的性能。需要记录的发生变化的信号数量和范围，以及记录的持续时间应该尽可能多地加以限制。当然，你不得不在这两者中折中：记录更多或更大范围的信号，或者多次运行仿真以增加解决问题所需要观测的信号。如果估计错误很有可能出现，例如，在设计开始阶段或在反复排错的过程中，则应该记录用来诊断问题的所有必需信号。如果估计没有错误，例如在测试运行期间，则不应记录任何信号。

不要使用波形浏览器来判断设计合格与否

在功能验证环境中，使用波形浏览器来判断设计的正确性时涉及到解释电脑屏幕上的几十条也可能是上百条波形与期望值的差异。对于十个以下的信号来说，这不失为一种可以偶尔使用的验证方法。随着信号数目和变化的增加，要进行正确性检查的信号关系也在增加，再算上仿真的时间和运行仿真的次数，功能错误被忽略的概率很快就会达到 1。

一些波形浏览器可以比较几组波形

一些波形浏览器可以比较两组波形：其中一组被设定为标准参考，以此检查另一组是否有不同。波形比较工具会标记出或突出显示任何差异。这个方法有两个重要的问题：

如何定义一组波形为标准参考（golden reference）

首先，标准参考波形是如何被定义为“标准”的？如果采用视觉检查，在大部分情况下，遗漏重大功能错误的几率等于 1。能真正得到标准波形的惟一机会是在重新设计时，需要保持周期精度的向下兼容性。但是，这样的设计非常之少。大部分设计在重新设计期间都进行了修改和提高，标准参考波形也失去了作用。

差异真的重要吗

其次，波形比较不是验证设计意图的正确抽象级别。有时，与标准参考波形之间的差异并不重要，也不是所有的输出信号在任何情况下都需要比较。重要的是信号变化的相对关系而不是它们的绝对位置。如果新的波形相对于参考波形仅仅偏移了几个时钟周期，比较器也会认为这种情况是错误。

2.5 代码覆盖

你是否忘记了验证代码中的一些功能

代码覆盖检查工具是一种能识别设计中的哪些代码在验证期间被执行过（更重要的是哪些代码没有被执行过）的工具。代码覆盖检查是一种在软件工程中使用了很长时间的方法。漏报错误（即不合格的设计被认为是合格的）的问题，就是不合格设计的表现与合格设计的表现

是一样的。通常不可能百分之百地肯定被验证过的设计在功能上是正确的。即使所有的测试平台都通过了仿真验证，是否还存在有可能引起功能错误、但却没有执行过的RTL代码呢？这就是代码覆盖检查能帮助你回答的问题。

首先要运行代码

图2.13说明了代码覆盖检查工具是如何工作的。首先要测量源代码，测量过程只是简单地在源代码的一些关键位置插入检查点以便记录这段代码是否被运行过。具体的测量方法因工具不同而各有差异，一些工具可能会使用语言的文件I/O特性（例如Verilog中的\$write语句或者VHDL中的textio.write过程），还有一些工具可能会使用仿真器本身的功能。



图 2.13 代码覆盖检查过程

不需要测量测试平台

只有待验证的设计源代码才需要检查覆盖率，目的是判断是否验证了所有代码。而测试平台的代码则不需要追踪来确认是否曾运行过。如果有一段重要的测试平台没有运行，这表示待验证的设计源代码的一部分没有运行。此外，有些测试平台代码只有当检测到错误时才运行，因此对于测试平台的代码覆盖率没有太大意义。

在运行时收集追踪信息

用所有可用的、不需要测量的测试平台来仿真测量代码，将所有仿真的追踪数据汇集到一个数据库，由这个数据库产生报告，评估各个验证方法的覆盖率。

语句覆盖就是块覆盖

最常见的覆盖是语句、路径和表达式覆盖。语句覆盖也称为块覆盖，块是连续排列的一系列语句。如果某一个语句被运行，块中的所有语句都将被运行。例2.11中的代码就是一个块的例子。只要if语句中的表达式为真，将运行名为acked的块中的所有语句，因此追踪该块的运行等价于追踪块中四个独立语句的运行。

例2.11 块和语句的运行

```
if (dtack == 1'b1) begin: acked
    as    <= 1'b0;
    data  <= 16'hzzzz;
    bus_rq <= 1'b0;
    state <= IDLE;
end
```

块的边界可能并不明显

块不一定都能轻易界定。在例2.12中，有两个块：一个在wait语句之前（包括wait语句），另一个在wait语句之后。wait语句可能一直没有结束而使整个过程一直处于等待状态，随后的语句可能都不会运行，因此它们构成了一个独立的块。

例2.12 被wait语句分隔开的块

```
address <= 16#FFED#;
ale     <= '1';
rw      <= '1';
wait until dtack = '1';
read_data := data;
ale      <= '0';
```

2.5.1 语句覆盖

是否运行了所有的语句

语句、行或者块覆盖率能衡量验证方法运行了整个代码中的多少行。通常，图形化的用户界面可以让用户浏览源代码并快速识别未运行的语句。图2.14是以图形化方式显示的一个调制解调器模型部分代码的语句覆盖报告。不同代码覆盖检测工具和源码浏览器的实际界面可能各不相同。

```
☒ if (parity == ODD || parity == EVEN) begin
☐   tx <= compute_parity(data, parity);
☐   #(tx_time);
☐ end
☒ tx <= 1'b0;
☒ #(tx_time);
☒ if (stop_bits == 2) begin
☒   tx <= 1'b0;
☒   #(tx_time);
☐ end
```

图 2.14 语句覆盖的例子

为什么部分语句没有运行

图2.14的例子说明八个可执行语句中的两个（25%）未被运行。为了使语句覆盖率达到100%这个理想的目标^①，首先要确定未覆盖的语句运行所需的触发条件。在上面的例子中，if

① 但是不一定能达到这个目标。例如，VHDL语言中的case语句包含了所有的可能情况后其default子句就不可能运行。

语句中的parity值必须是ODD或EVEN。一旦触发条件确定,就要检查这个条件为什么没有发生:这是一种不可能发生的条件吗?这是应该被现有的验证方法检验的条件吗?或者这个条件被忽略了?

一些语句没有运行是正常的

如果这个条件不可能发生,则与之相关的代码实际上是多余的:它永远不可能运行。明智的做法是删除这段代码,这可以减少源代码的混乱并增加其可维护性。但是,一个好的具有保护性的(包含一切可能情况的)代码通常包含此类不被运行的代码。这些附加的代码可以监控并报告那些不允许出现的情况,从而证明假设是错误的。这种附加的代码是很有效的(参见2.8节)。它可以在有问题的源代码附近识别出功能错误,而不用依赖于你在找寻其他问题时这段代码恰好产生一个意外的应答。

你的模型可以告诉你结论是否和假设的一致

例2.13所示的是一个可综合的case语句保护性建模的例子。尽管综合控制命令能告诉综合工具case语句列举了所有可能的情况,但是在仿真过程中有可能会产生预期之外的状态,导致仿真结果可能与实际硬件产生的结果不同,而且很可能到门级仿真或者器件失效时才会发现这种不同。

例2.13 保护性编程技术

```
case (mode[1:0]) // synopsys full_case
2'b00: ...
2'b10: ...
2'b01: ...
// synopsys translate_off
// coverage off
default: $write("Case was not really full!\n");
// coverage on
// synopsys translate_on
endcase
```

检查代码覆盖率时应把保护性的语句排除在外

在进行代码覆盖检查时,应该找出保护性的代码语句并将它们排除在代码覆盖统计之外。例2.13中的注释就是用来实现以上功能的。有些代码覆盖检查工具可以设置成自动忽略综合控制命令(translation on/off)之间的任何语句,如果将其设定为确保translation on/off命令之间的代码不被运行,将更有意义。

加入测试用例以运行所有语句

如果在验证期间,有些代码本来应该运行但却没有运行,说明一个或者多个测试平台是错误的或者不完善的。如果发生了这种情况,那么应该再加入已经编写好的测试平台、编写新的测试平台、或者用不同的初始条件再多次运行仿真。

2.5.2 路径覆盖

运行一组语句的路径通常不止一种

路径覆盖用来检查一组语句运行的所有可能路径。例 2.14 中的代码有四种可能的路径：第一个 if 语句的条件可能为真也可能为假，第二个也是如此。为了验证这个简单代码的所有路径，必须运行两个 if 语句的所有可能值：即假 - 假，假 - 真，真 - 假，真 - 真。

例2.14 语句和路径覆盖的例子

```

if (parity == ODD || parity == EVEN) begin
  tx <= compute_parity(data, parity);
  #(tx_time);
end
tx <= 1'b0;
#(tx_time);
if (stop_bits == 2) begin
  tx <= 1'b0;
  #(tx_time);
end

```

为什么某些语句没有运行

上图采用的验证方法表明，尽管它提供了 100% 的语句覆盖率，但只提供了 75% 的路径覆盖率。因此需要判断未被覆盖的路径的运行条件。在这个例子中，parity 必须既非 ODD 也非 EVEN，并且停止位等于 2。接下来的问题就是这个条件是否可能发生？是否被忽略了？

应该限制语句序列的长度

语句序列的路径数目与控制流语句的数目成指数关系。如果一段代码序列的路径数目过大，代码覆盖工具将放弃计算它的路径覆盖率。为了避免发生这种情况，应该将所有的顺序代码结构（在 Verilog 中：always 和 initial 块、task 和 function；在 VHDL 中：process、procedure 和 function）保持在 100 行之内。

要达到 100% 的路径覆盖率是非常困难的。

2.5.3 表达式覆盖

导致控制流变化的原因可能不止一个

如果仔细观察例 2.15 中的代码，会发现有两个相互独立的条件可以触发第一个 if 语句运行 then 子句：parity 被置为 ODD 或者 EVEN。例 2.15 所示的表达式覆盖测量了代码运行时的不同路径。在这个例子中，语句覆盖率达到了 100%，但表达式覆盖率只有 50%。

例2.15 语句和表达式覆盖的例子

```

if (parity == ODD || parity == EVEN) begin
    tx <= compute_parity(data, parity);
    #(tx_time);
end
tx <= 1'b0;
#(tx_time);
if (stop_bits == 2) begin
    tx <= 1'b0;
    #(tx_time);
end

```

除此之外,有必要弄清楚表达式的控制条件为什么没有运行?在上面的例子中,测试平台没有将 parity 置为 EVEN。这个条件不可能发生,还是被忽略了?

达到 100% 的表达式覆盖率是极其困难的。

2.5.4 有限状态机覆盖

语句覆盖能检测到未访问的状态

通常有限状态机 (FSM) 的每个状态都明确地与 case 语句的一个分支相对应,因此任何未被访问的状态都可以通过未覆盖的语句清楚地识别出来。验证过程中一个未被覆盖的 case 语句对应了一个未访问的状态。

FSM 覆盖能识别状态转移

图 2.15 所示的是一个有限状态机的状态转移图。虽然它只有 5 个状态,但可能的状态转移路径却很多:在相互连接的状态之间存在 14 种可能的状态转移路径。按照 Reset、A、B、D 和 C 状态的顺序可以轻易地达到 100% 的状态覆盖率,但这仅仅达到了 36% 的状态转移覆盖率。要完整地验证这个有限状态机的运行,必须保证所有预期的状态转移路径都被运行。

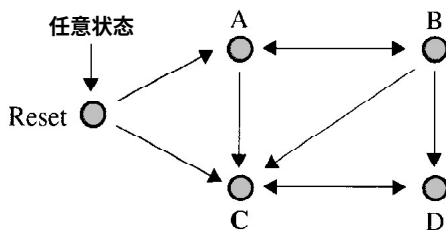


图 2.15 FSM 的状态转移图

不确定的状态会怎么样

图 2.15 中的有限状态机只显示了 5 个确定的状态。一旦综合到硬件,最少需要三位的状态寄存器 (如果采用其他状态编码方案,例如 one-hot, 可能需要更多的状态寄存器), 这样会产

生三个可能的不确定状态。如果辐射干扰使设计进入了这三个不确定状态中的一个,那会怎么样?设计的正确性会因此受影响吗?EDA工具的逻辑优化过程可能使解码逻辑在三个不确定状态之间形成孤岛,状态转移只在这三个状态之间进行。只有当 reset 发生时,设计才能从这三个状态恢复为确定的状态。设计的安全、可靠性以及保证这些的技术问题不在本书的讨论范围之内,但这是验证工程师需要考虑的问题。

2.5.5 100% 的代码覆盖率意味着什么

完整不代表正确

对这个问题的最简单的回答是:你写的每条语句都被运行了。代码覆盖率说明的是验证方法执行了多少源代码,但这并不能以任何方式说明验证的正确性和完整性。图2.16展示了自动提取的代码覆盖率的恢复模型,它清楚地说明了代码覆盖率并不能帮助验证设计的意图,只能表明 RTL 级代码是否被完全运行过。

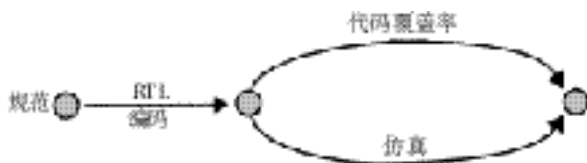


图 2.16 自动代码覆盖率的恢复路径

应该持怀疑态度来分析代码覆盖检查工具的结果。它们可以帮助找出验证中未运行过的死角或者与工艺相关的特性,也可以用它来判断未被覆盖的情况是值得引起注意的、还是无关紧要的。

代码覆盖让你知道验证工作是否还未完成

低的代码覆盖率表明验证工作还没有完成,但高的代码覆盖率并不表明验证工作已经完成。代码覆盖率只是验证工作完整性的一种标志,它可以增加对验证工作完整性的自信心,但不能作为验证工作完整性的惟一标志。

代码覆盖工具可以作为剖析工具

当开发专门用于仿真的模型时,它的性能成为重要的衡量标准,代码覆盖工具可以用来剖析该过程。剖析的目的与代码覆盖检查的目的恰恰相反,它是用来识别运行最多的代码行,而这些代码行是性能优化的首选。

2.6 功能覆盖

你是否会忽略验证一些情况

功能覆盖检查工具是一种避免不合格的设计通过测试平台验证的工具。虽然这种方法已经在一些公司使用过很长时间,但是直到最近它才被加入到通用验证工具库。功能覆盖记录的是一些相关的数据(例如数据包长度、指令码、缓冲器占用水平),以保证验证过程用所有的期望值验证了设计。代码覆盖衡量多少代码被执行过,而功能覆盖则衡量设计的原始要求实现了多少。

功能覆盖是对代码覆盖的补充

高的功能覆盖率并不代表高的代码覆盖率。代码覆盖关心的是如何记录代码的执行,而功能覆盖关心的是所执行的功能的意图或目的。例如,CPU指令的解码可以由不同的 case 语句来对应操作码中的每个字段。通过对操作码各个字段的各种组合译码,每个 case 语句可能有 100% 的代码覆盖率。但是,对特殊 CPU 指令的译码仍然有可能没有执行。

功能覆盖可以检测出忽略的错误

例 2.16 所示的是一个 CPU 指令译码的 case 语句,我们可以发现 RTS 指令的译码是如何被遗漏的。如果仅仅依赖代码覆盖率,由于覆盖率达到了 100%,可能会给人一种代码是正确的假象。代码覆盖工具要报告一段未被覆盖的代码的首要条件是该代码必须存在。而功能覆盖检查则不依赖于实际的代码,它通过检测代码是否处理了期望值来报告覆盖率。

例2.16 代码覆盖工具未检测到错误的代码

```
type OPCODE_TYP is [ADD, SUB, JMP, RTS, NOP];  
...  
case (OPCODE) is  
  when ADD => ...  
  when SUB => ...  
  when JMP => ...  
  when others => ...  
end case;
```

必须由人定义

由于代码覆盖工具的低使用成本,很快地被应用到验证过程中。它们几乎不需要用户的任何额外操作,最多只是在编译代码之前多执行一个命令。而由于功能覆盖涉及到测量一些相关值,所以必须进行一些人工操作。因为值的相关性和重要性是与设计意图相关的,所以功能覆盖是不能从 RTL 级源代码自动提取的。功能覆盖率的高低取决于如何实现它。

在运行过程中收集覆盖率数据并分级

与代码覆盖类似,功能覆盖数据也在仿真运行过程中采集。每次运行过程的数据都被收集到一个数据库或者单独的文件中,然后这些独立运行的功能覆盖数据被汇总到脱机的分析器。

所有独立运行的边缘覆盖率数据将被分级,从而判断哪次运行对整个功能覆盖率目标的贡献最大。这些贡献大的运行过程将被赋予优先级,而其余的将被忽略。

可以在运行中使用覆盖数据

功能覆盖工具通常提供一些进程,测试平台可以靠这些进程动态地查询某项功能覆盖率,然后由这些信息来修改自己的行为。例如,它可以增加生成未覆盖值的概率,还可以在连续两次查询的结果表明覆盖率没有显著增加时自动停止仿真。

虽然功能覆盖检查工具被销售商们吹捧为一个有力的工具,但它不是万能药。执行动态反馈机制并非易事:你必须使你的激励产生过程与功能覆盖率相关联,并且保证激励产生过程以功能覆盖率为目标。当输入数据和被测量的覆盖率之间有直接联系时,例如指令类型,动态反馈的工作效率最高。采用简单的测试平台,不使用动态反馈而只是多运行几遍,比使用相对复杂的测试平台的单次运行能更有效地达到目标。

2.6.1 项目覆盖

是否生成了所有的重要和相关数据

项目覆盖用来记录单独的项目数据,它是所有功能覆盖工具的基础。它的目的是要保证设计内部、输入以及输出的所有重要和相关的都被观测到。项目覆盖的例子包括但不限于信息包长度、指令操作码、中断信号级别、总线事务结束状态、缓冲器占用水平、总线请求模式等。

定义取样内容

记录功能覆盖非常简单,但容易带来大量的覆盖数据。数据并不等同于信息,因此应该将覆盖检查限制在反映设计验证程度的数据上。例如,如果关心的是基于指针值的缓冲器空间的利用率,那么检测FIFO中的读写指针值就足够了。但是如果观测的是FIFO的占用水平(FIFO是否为空?是否为满?是否溢出?),那么就必须测量并记录指针值之间的差异。

定义取样点

接下来,就要决定在测试平台或者设计中的哪个位置进行取样,以得到准确和相关的测试数据。例如,可以在以下几处对指令的操作码取样:代码发生器的输出端、存储器接口、译码寄存器处或者执行流水线里。无论在何处取样,都必须保证记录的测试数据是覆盖率工具需要的数据。

例如,如果要测试被执行过的操作码,那么这些操作码就应在执行单元中取样。如果在译码单元中取样,当分支指令或异常造成译码流水线刷新时,就可能导致错误的取样值。同理,

在包发生器的输出端取样信息包的长度也可能导致错误的取样值：如果信息包在传输过程中被测试平台中的底层函数注入的干扰破坏，信息包就可能被丢弃。

定义取样时刻

测试数据是在仿真中的一些时间点被取样的，这些时间点可以在每一个时钟周期内、每当地址控制信号有效时、请求被允许时、或者在随机产生一个新数据之后。选择取样时间点必须谨慎，因为过度的取样可能降低仿真的性能、消耗数据库资源而没有得到有用的附加信息。

取样的数据必须是稳定的，所以应该避免取样数据和取样事件（参见4.8.1节）之间的竞争。为了减小瞬态值被取样的概率，功能覆盖工具可以将取样延迟到仿真周期的末尾，下一个仿真周期之前（参见4.7.5节和4.7.6节）。

高层测试平台函数通常在每个仿真周期的零时刻进行数据操作。如果功能覆盖检查工具将取样时刻延迟至仿真周期的末尾，就可能无法取样到所有的中间值，而只记录了最终值，导致取样率不足。

说明检查的原因

如果功能覆盖是用来测试重要和相关数据的话，那么说明这些数据重要和相关的原因是很必要的。例如，测试一个32位地址的功能覆盖可能导致采集超过40亿的数据。虽然这些数据在数值上不同，但在功能上却是等价的，因此可以将那些功能上等价的数据归入同一类，这样可以把数据的数量减小到可以处理的程度。例如，对于译码电路来说，地址0x00000001到0x7FFFFFFF和地址0x80000000到0x8FFFFFFF在功能上是相同的，所以可以将数据减少到四类（最小值，1到中间值，中间值到最大值减1，最大值）。

功能覆盖可以检测出错误数据

既然可以将功能相同的数据分类，那么同样也可以将错误或预期值之外的数据分类。功能覆盖可以作为检错工具来使用，就像测试平台代码中的if语句一样。但是，也不能仅仅依赖于功能覆盖工具来检测错误数据。功能覆盖工具只是运行时的一个选项，并不一定在所有仿真时间内都工作。如果为了提高仿真性能而关闭了功能覆盖检查，并且某个数据只在功能覆盖工具内被定义为错误，那么这个错误值很可能检测不到。

功能覆盖可以报告漏洞

功能覆盖检查的最终目的是识别代码中不完善的地方。在分析中，功能覆盖检查工具可以比较包含至少一个样本的测试数据类的个数和总测试数据类的个数。任何没有样本的测试数据类都是功能覆盖检查结果报告中的漏洞。通过检查这些空类，设计者可以在测试中关注这些漏洞并在验证中很快地完善它们，而不是重复地执行那些已经验证过的功能。

测试数据类的总数必须相对较小,否则就难以检查这些空类。例如,如果没有明确的分类,就不可能测试 32 位数据的覆盖率,因为漏洞的数目可能成千上万,使检查成为不可能。应该尽可能地限制测试数据类的数量,例如 Specman Elite 对于漏洞检查默认限制最多有 16 个类。

2.6.2 交叉覆盖

是否产生了所有重要值的组合

项目覆盖关心的是独立的值,而交叉覆盖测试的是各种值的组合。它可以回答如下的问题:“是否在所有的端口插入了错误的信息包?”“是否执行了所有操作码和操作模式的各种组合?”“当缓冲器为满、不空和空时,状态机是否经历了每个状态?”交叉覆盖可以包含两个以上的项目。但是,测试值数据的数量将随着交叉项目的数目成倍增长。

与项目覆盖相似

在机理上,交叉覆盖类似于项目覆盖。值是在特殊位置或特殊点、在测试取值范围内采样得到的。惟一的不同点是两个或者更多个值同时被采样而不是项目覆盖里的一个值。

可以在后处理步骤中完成

在脱机的情况下,可以通过对项目覆盖的后处理来获得交叉覆盖。有些工具只能通过这种方式得到交叉覆盖。为了能进行脱机的交叉覆盖分析,在对每个单独的值采样的同时,必须记录采样时刻。这些记录的关于仿真时间的附加信息增加了总的数据量,也降低了仿真的性能。因此大部分工具都把记录附加的交叉覆盖信息作为一个选项,并且这个选项通常默认为关闭。如例 2.17 所示,交叉覆盖报告是通过同一仿真时刻的采样值进行分析得到的。

例2.17 脱机的交叉覆盖分析

Packet Length	Packet Valid	Length x Valid	

short @10	good@10	good	bad
long @20	bad @20	short	X
medium@30	good@30	medium	X
		long	X

脱机的交叉覆盖报告可能导致错误的样本

只要每一个项目值是在不同时刻采样的,就能进行正确的脱机交叉覆盖分析。当记录值位于 RTL 设计或者总线功能模型之中、在时钟沿被采样时,在每一个仿真时刻就只有一个值。但是在高层测试平台的零时刻,可能会有多个值在同一仿真时刻被采样。例 2.18 中列出的是与前面一样的项目覆盖,但是是在所有信息包同时产生时进行采样。这个方法会导致不正确的脱机交叉覆盖报告。而如果是在运行过程中采集数据,就能得到正确的结果。

例2.18 错误的脱机交叉覆盖分析

Packet Length	Packet Valid	Length x Valid		
short @10	good@10		good	bad
long @10	bad @10	short	X	X
medium@10	good@10	medium	X	X
		long	X	X

2.6.3 转移覆盖

是否产生了所有的重要值序列

交叉覆盖关心的是同一时刻的值的组合,而转移覆盖测试的是值的序列。转移覆盖可以回答以下问题:“是否执行了所有相邻读写周期的组合?”“是否执行了所有测试操作码之前的算术操作码的组合?”“该状态机是否遍历了所有重要的路径?”转移覆盖可以涉及同一个项目的两个以上的连续的值,但数据的数量会随着状态转移的数量成倍增加。

与项目覆盖相似

在机理上,转移覆盖与项目覆盖相似。值是在特殊位置或特殊点、在测试取值范围内采样得到的。其不同点是样本值是在两个或多个连续的项目样本之后采样得到的。另一个不同点是转移覆盖样本可以重叠,两个转移覆盖样本可以由同一个项目样本构成。

与FSM 路径覆盖相似

在概念上,转移覆盖与FSM 路径覆盖相似(参见2.5.4节),两者都记录了特殊位置的连续值(例如,一个状态寄存器),并且都比较了可能的路径序列。但是与FSM 覆盖工具不同的是,FSM 覆盖工具仅限于检查RTL 代码中的状态寄存器,而转移覆盖工具可应用于测试平台和行为级模型中的任意样本值。

转移覆盖能反映设计意图

由于转移覆盖是根据设计意图和实现人工设置的,因此它能提供一种独立验证设计正确性和完整性的方式,它能检测出错误的转移,识别出设计时忽略的转移。

2.6.4 100% 的功能覆盖率意味着什么

它反映的是完整性,而不是正确性

功能覆盖反映的是哪些重要和相关的情况被验证过,它表示了验证计划执行的完整性。除非将某些数据的集合定义为错误,否则功能覆盖就无法判断某个状态或设计对某个状态的反映是否正确,所以说功能覆盖的好坏取决于所定义的功能覆盖模型。100% 的覆盖率意味着在仿真中已经涵盖了所有的覆盖点,它与你的功能覆盖模型的完整性无关。

应该持怀疑态度来分析功能覆盖检查工具的结果。由于这些结果是由额外的测试平台代码产生的，因此需要经过调试并验证其正确性，这样才可以识别那些未包含在验证计划中的状况。

如果某项覆盖并不重要，就不要计算它

计算功能覆盖率会产生许多相关的报告，如果覆盖率测试没有明确的目标，很快就会产生数兆字节的功能覆盖率报告。这些报告中的绝大部分与100%的覆盖率无关，导致你无法判断重要的报告和重要的漏洞。验证计划（参见第3章）应该作为覆盖模型和验证环境的功能规范。如果某个报告并不重要或者没有特别的意义，在仿真之后也不需要去检查它，那么它存在的价值就值得考虑。

代码覆盖让你知道验证工作是否已经完成

如果使用得当，功能覆盖检查工具可以作为验证计划的正式规范。一旦达到了100%的功能覆盖率，就意味着已经执行过预先定义的所有重要和相关状态，即完成了所有的验证计划任务。但是，这和验证计划本身的完整性或正确性无关。

2.7 验证语言

VHDL 和 Verilog是仿真语言，不是验证语言

Verilog的重点是描述低级硬件结构，在Verilog-2001中刚刚加入了对于基本的高级数据结构的支持。VHDL是面向大型设计的语言，所有信息都要通过严格定义的接口来通信。这些限制阻碍了验证计划的有效执行，同时VHDL和Verilog还缺乏一些有效执行现代验证过程的重要功能。

验证语言能提高抽象级别

如第1章所述，提高设计效率的途径之一就是提高设计的抽象级别。高级语言，如C、Pascal是从汇编语言抽象出来的，使软件工程师的编程效率大大提高。类似地，专门面向验证的计算机语言比通用的仿真语言抽象级别要高。硬件验证语言具有一些与硬件设备交互的重要概念：定时、并发、例化。它们还具有提高抽象级别的特点：复杂数据类型、面向对象、时序断言。

验证语言能使验证自动化

如果硬件验证语言的主要优点仅仅是高抽象级别和面向对象的话，那么C++语言可能早就成为最好的代替者了^①。C++是免费的，并且已经在广泛使用。如图2.17所示，HVL语言的主要优点是它们可以随机产生激励来自动完成一部分验证，同时收集功能覆盖来识别漏洞，还可

^① C++ 仍缺乏定时、并发、例化的概念。

以加入约束来产生更多的激励以填补这些漏洞。为了支持这些功能，一些HVL语言提供了带约束的随机发生器、功能覆盖测试和代码扩充功能。

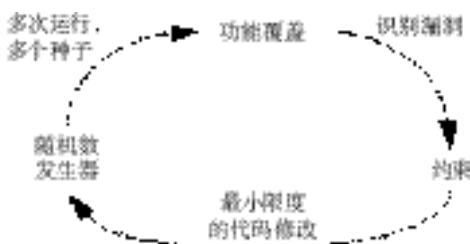


图 2.17 HVL 语言工作流程

存在多种验证语言

在编写本书时，商用的验证语言工具包括Verisity公司的e语言，Synopsys公司的OpenVera语言，Forte公司的RAVE语言。开放源代码或公开性语言包括：Cadence公司的SystemC Verification Library^①和Juniper Networks的Jeda。在SystemVerilog中引进了HVL功能的Accelerate。另外还有很多个人开发的基于C++、Perl和TCL的解决方案。

现在还没有关于HVL语言的明确定义，上面提到的大部分语言都没有包含HVL语言工作流程中的所有功能。大部分的验证语言工具只是弥补了Verilog和VHDL在验证方面的不足，它们还是一个不成熟的市场，未来可能会出现一两个成熟的市场占有者，并形成实际的行业标准。

在学习验证语言之前应掌握验证的基本要素

在本书中，作者将VHDL和Verilog语言作为描述验证基本结构的语言，以此引入自检验的事务级直接测试平台的概念。虽然使用HVL能更容易地编写测试平台（特别是自检验部分），但仍需要对验证进行规划、定义验证的目标、方法、结构、产生激励、判断预期响应并比较实际输出。这些都可以通过VHDL和Verilog实现。

覆盖率驱动的受约束的随机方式需要HVL语言

所有的HVL语言都可以像使用Verilog或者VHDL语言那样来使用，实际上，大多数HVL语言就是精简的Verilog或VHDL。但如果使用的HVL语言包含了支持验证流程的所有功能，那么它的验证过程就要以不同的方式来执行。

这种变化就像使用逻辑综合工具一样，它使用一种不同于原理图的输入方式。为了成功地运行覆盖率驱动的受约束的随机验证方式，需要修改验证计划、验证方式和测试用例。由于Verilog和VHDL缺少以上功能，所以后面将采用e和OpenVera阐述这些概念。

① 它的前身是TestBuilder。

2.8 断言

断言（Assertion）能检测出错误的状态

断言就是一些特殊的if语句，当该语句的条件不成立时将产生错误信息。断言已在软件界使用了多年：assert()函数就是ANSI C标准的一部分，它用来检测空指针、空序列之类的情况。VHDL同样带有assert语句，但它并不常用——常用来作为结束仿真的条件（参见5.4.2节的例5.44）。

硬件断言需要能够描述时序的语言

软件断言只是在“assert”语句执行时判断其条件是否为真，这种简单的零延时测试方法对于硬件设计的断言来说是不够的。在硬件中，功能正确性通常包括一段时间内的行为。“该状态寄存器是one-hot编码”、“该FIFO从不溢出”等诸如此类的硬件断言可以作为零延时的表达式，但是“该信号必须在一个时钟周期后赋值”或“请求必须在十个时钟周期内处理”这样的硬件断言就需要带有延时信息的断言条件。因此，断言需要能够描述时序的语言来描述这些延时关系。

有两类断言

断言被分为两大类：一类是由设计者定义，另一类由验证工程师定义。

- 由设计者定义的设计断言（Implementation Assertion）
- 由验证工程师定义的规范断言（Specification Assertion）

设计断言用来验证假设情况

设计断言一般用来指示设计中的误用或故障情况。例如，可以在FIFO中加入断言语句来检测其是否发生溢出、下溢；或者由于设计的缺陷导致write和read脉冲同时有效。因为设计断言是由设计者定义的，所以它不能用于检查设计意图和实现的功能之间的差别，但可以检测出设计的实现情况和设计假设情况之间的不同。

规范断言用来验证设计意图

规范断言通常用来检查设计的功能是否达到预期目标。这类断言语句相当于功能错误的检测器，它可以弥补测试平台的自检验部分的不足。规范断言是典型的白盒验证（相对于黑盒），因为现代设计的主要输入和输出之间的关系太复杂，很难用某种时序描述语言来描述。例如，与其依靠记分板来检测一个判决器的性能，不如使用更简单的block级断言来进行检查。

断言规范是一个复杂的课题

这里仅仅对断言做了简单的介绍，它有着复杂和强大的功能。本书的其他章节将详细介绍断言。

2.8.1 仿真断言

OVL引起了一场使用断言的风暴

断言几乎是以风暴式的速度进入到硬件设计的大家族的，这起源于 Foster 和 Bening 的一本书^①，它通过使用实现了各种设计断言的 Verilog 预定义模块库的方式引入了这个概念。这些库的源代码可以在 Open Verification Library^②中查到，它是使用 Verilog 来描述时序的一种好方法。接下来，Foster 在 Hewlett-Packard 致力于这样的工作：让设计者在工作中使用断言并证明这是行之有效的方法。用 Verilog 的模块使设计者能很方便地采用断言，随后的实践也证明了该方法是富有成效的。

断言能检测出在时间和空间上与实际故障非常接近的错误

如果设计中的假设情况在仿真中不成立，那么这个设计就不正确。查明假设条件不成立的原因并不重要：原因可能是设计者对于某一个 block 的误解，或者其他相关 block 设计者的错误，也可能是由不正确的测试平台引起的。总之结果是导致设计工作的情况与最初的目标不一致。而低级的故障症状通常不易被发现（不是所有情况），除非有关的数据直接影响输出或者该症状被自检验结构发现。

关于设计假设的断言一般都在故障发生时立即报告其发生时间及位置。调试和修改断言报告的错误将比查询有问题的信息包效率更高，在 Foster 的一个项目中，85% 的设计错误都通过仿真断言迅速地找到并给予了即时的修改。

2.8.2 形式断言证明

断言有可能发生吗

仿真能显示出错误的存在，却不能证明它们的不存在。即使在一系列仿真中断言未报告错误，也不能证明错误不可能发生。代码覆盖和功能覆盖检查工具可以告诉我们设计的某部分被完整的验证过，但是我们对设计的正确性仍应该持怀疑态度。

模型检查能从数学上证明或否定断言

称为“模型检查器”或“断言证明器”的形式验证工具能从数学上证明这样的情况：给出一个 RTL 设计和输入信号之间的假定关系，断言条件就一直成立。以一个计数器为例，形式

① Harry Foster 和 Lionel Bening 的“Principles of Verifiable RTL Design”，second edition, Kluwer Academic Publisher, ISBN 0-7923-7368-5。

② 见 <http://verificationlib.org>。

验证工具能够找出导致断言报告错误的事件序列。形式验证工具提供关于设计的附加信息，设计人员的任务是判断这些事件序列是否可能发生。

有些断言被用做假设条件

如果设计的输入信号是任意的，那么所有断言的条件都有可能不成立。好在为了保证设计的正常工作，每个设计的输入信号都是有一定限制和规定的。而且，这些输入信号往往来自于可以正常工作的其他设计。所以检验设计断言时，提供一些作为输入和状态的断言是很有必要的。这些输入和状态的断言没有被证明过，但它们被视为成立并作为验证结论的约束条件。

输入信号的假设也需要证明

验证结论的正确性取决于设计输入信号的假设^①的正确性。如果这些假设是错误的，那么结论就不可能成立。因此设计输入信号的假设也成为一种断言，应该在设计被验证之前进行检验。

半形式的工具将模型检查和仿真融合在一起

半形式的工具将形式的方法和仿真混合在一起，这是一种缩短常规方法（仿真）和形式验证工具之间差距的尝试。它们使用仿真的中间信息，例如设计的当前状态等，作为检验断言的起点。

用形式的方法来检验仿真中未覆盖的情况

形式的验证方法并没有代替仿真。相反，仿真（包括仿真断言）才是验证的重要工具：它仍是检验功能覆盖情况的最好工具，并且能够排除各类明显或不明显的错误。形式验证方法在设计的重要部分的关键处插入一些标志点来测试验证工作的进行情况。根据仿真收集的功能覆盖率（例如判决器的各种请求模式）可识别出有待进一步验证的状态。如果这些状态难以由仿真环境产生，那么可以将这些状态作为假设，来证明设计中未覆盖状态的正确性。

2.9 版本控制

被验证的是否是同一个设计

验证工作的一大难题就是要保证待验证和被验证的是同一个设计。当编译一个 Verilog 源文件时，如何保证设计人员在综合时使用同一个文件呢？

当验证和综合由同一名设计人员完成时，只要他对文件进行了有效的管理，这个问题发生的可能性就较小。但是，在实际工作中，如第1章所述，在功能验证过程中由同一名设计者完成这两项工作是不可靠的。通常是由不同的设计者分别进行验证和综合。

^① 形式验证领域将这些输入断言称为约束（constraint），这里作者使用术语“假设”（assumption）将其与随机发生的约束区分开来，后者是一种随机的概念。

文件必须集中管理

在小型设计队伍中,有可能所有设计者都在同一个目录下工作,也有可能将设计文件分布在少数几个独立的目录下,每个人管理自己的目录同时了解其他人的文件位置。这种情况很普遍也很危险:你很难了解其他设计者是否修改了源文件?是否在你刚刚验证完后又产生了其他的功能性错误?

在同一位置获取文件是很方便的

这种方法不具备可扩展性,一旦设计组增加到两三个人以上就难以实现了。而当设计人员的工作环境不在同一地理位置时,这种方法就完全不适用了。验证工程师是这个问题的主要面对者。一个被恰当划分的设计一般不需要涉及到其他人目录下的资源,设计者只需要独立地工作在自己的目录下,而验证工程师的首要工作就是将这些零散的设计组合成一个整体。显而易见,把分散在多个文件服务器上的不同工作环境下的零散文件组合起来是很困难的工作。

2.9.1 软件工程经验

HDL 模型的管理实际上是软件管理的课题

在过去的 25 年中,软件工程一直在解决的一个问题就是如何管理由不同设计者创建和验证的源文件,并最终组合成一个完整的产品。毫无疑问,管理基于 HDL 的硬件设计和管理一个软件项目完全一样。

有一些免费工具和商用工具

软件工程师们用源码控制系统来管理文件,其中一些是免费的软件,工作于 UNIX 操作系统下(RCS、CVS、SCCS),或由 GNU 项目发布(RCS、CVS),可以从以下网址找到相关的源代码:<ftp://prep.ai.mit.edu/pub/gnu>。也有相应的商用系统,但往往都非常复杂。

所有的源文件都被集中管理

图 2.18 展示了源文件是如何通过源码控制系统来进行管理的。所有对于源文件的访问和修改都要通过管理系统这个中间媒介,设计者和用户只与管理系统交互信息,而不是直接访问工作目录下的文件。

对文件所有的操作都被记录下来

源码控制系统不仅仅保存文件的最新版本,还完整地记录了每个文件的各个版本。因此,通过它可以恢复文件的旧版本,还可以判断出两个版本之间的差别。不断地更新文件的版本是个不错的主意,因为这样就不用依赖文件备份系统来恢复那些意外删除的文件。当对设计所做的一系列修改没有见效时,可以通过源码控制系统将设计恢复到修改前的版本。

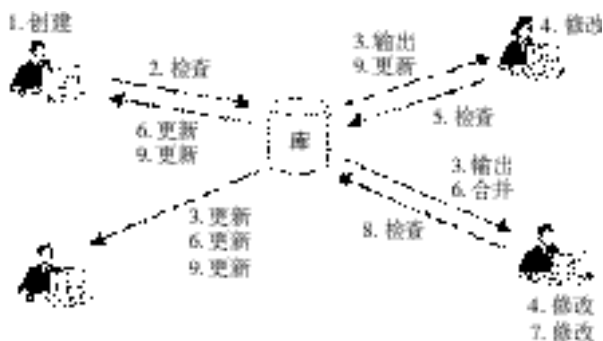


图 2.18 源码控制系统中的数据流

设计团队拥有所有的文件

使用源码控制系统时,文件不再由单个的设计者所拥有。设计师可能在名义上占有设计中的某些部分,但具有适当权限的任何人都可以对任何文件进行修改。这使得验证工程师可以很快地修改 RTL 代码中的错误,而不用等待原来的设计师去修改。源码控制系统通过使用文件锁或者合并修改来管理文件的修改。

2.9.2 配置管理

不同用户通过源码控制系统得到的文件版本可能不同

设计中的源文件（或其中的一部分）对于每个通过源码控制系统工作的设计师都有独立的视图。图2.19说明的是两个用户如何从管理系统中得到源文件的不同版本。视图不一定由设计中所有文件的最新版本组成。事实上,这可能成为验证工程师的工作障碍。由创建者输入的文件可能包含有语法错误、有待进一步修改、或者是个完全错误的设计,如果文件更新速度快于验证过程发现错误的速度,那么这将给工作带来很大的问题。

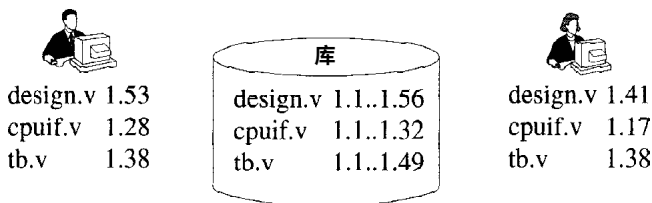


图 2.19 用户观察到的被管理的源文件

配置是通过给不同版本的文件附加标记符而形成的

所有的源码管理系统都有符号标记符的概念,用来区分不同的文件版本。用户可以使用符号名称来获取相应的文件或文件集,而不用涉及该文件的具体版本编号。在图2.19中,左边的

用户可能正在使用设计者定义的“待仿真”的版本，而右边的系统验证工程师可能正在使用ASIC验证工程师定义的“golden”版本。

配置管理向符号管理的转换

发行版本的管理归根到底就是标记符管理，这是一项复杂的工作。表2.1中列出了在设计中使用的一系列标记符，在设计过程中用做识别一个文件的不同版本。类似“Version_M.N”这样的标记符一旦赋给某个版本的文件后就不再变化，而另一些类似“Submit”的标记符随着设计过程的进行不断更新。在更换标记符之前，最好能保留该标记符前一位的信息，这可以通过在标记符名称中附加日期来实现。例如，旧的“Submit”版本被新的标有“Submit_000302”（于2000年3月2日修改）的标记符所替代，并更新为最新的版本。

表2.1 发行管理的标记符的例子

标记符名称	说明
Submit	等待进行功能验证，设计者已经完成语法正确性和初步功能性的验证
Bronze	通过了基本的功能测试用例，可以用于系统集成
Silver	通过了所有的功能测试用例
Gold	通过了所有的功能测试用例并且达到了预定的代码覆盖标准（还需要边界情况的测试用例）
To_Synthesis	等待进行综合。通常已经达到了“Silver”或“Gold”
To_Layout	等待进行版图布局设计。通常已经达到了“Gold”
Version_M.N	用于制造加工的版本。已经达到了相应的“To_Layout”的标准。同一设计的后续版本以此为参考更新标记符
ON_YYMMDD	带有具体日期的属性

2.9.3 发行版本的管理

当文件的新版本被输入到源码管理系统的数据库并改变了标记符时，以前的视图就过期了。

发行版本是一种特定的配置

设计中某部分RTL代码的设计者一般总是使用该文件的最新版本，并不断地修改和更新它们（通常是在代码开发过程中的某些关键点和每个工作日结束时）。一旦源码通过了语法正确性测试并且功能达到了设计要求（通过一些特定的测试平台），那么该文件的相应版本就被标记为“等待验证”。

用户必须将文件更新至合适的发行版本

作为一名验证工程师，你必须不断地观察并及时更新文件的版本。有可能花费好几天时间来编写一个特别复杂的测试平台，以至于没有及时将待验证的设计文件更新到最新版本。这样，可以对待测试的设计保持有连续的思路，并不断修改测试平台。但只要真正的验证和调试工作一开始，在运行这个测试平台之前必须将文件更新至最新的“等待验证”发行版本。

随时更新

当多个设计者同时使用同一个文件进行设计时,应该经常检查文件并不断更新,以合并对文件的并发修改。如果停滞时间过长,很可能会需要人工干预来解决更新冲突。这种对文件并行修改后立即合并更新文件的方法看似冒险,但实际经验表明,不同的函数或错误修正极少位于源代码的同一行内。只要这类修改被两三行未修改的代码分割开来,合并操作就没有任何问题。并行开发是一种发展方向。

可以通知设计者新版本发布

一些源码管理系统的一个重要特性就是可以通过发电子邮件的方式告知设计者何时发生了重要的事件。例如,当“等待验证”的标记符发生变化时,系统就给所有的验证工程师发电子邮件,在电子邮件中还可以有选择地包含一些对于源文件变化描述的信息。由此用户就可以判断是否要立即更新当前的文件版本。

2.10 问题追踪

所有错误都交由验证工程师处理

验证工程师的工作是要找出错误。一般情况下都会找出一些异常的功能,如果没有发现问题,反而值得怀疑。因为存在问题是非常普遍的情况,这并不能反映硬件设计者的能力或水平。即使是最有经验的软件设计者写的代码也会有错误,简短的程序也不例外。既然可以肯定会找出错误,那么应该如何处理它呢?

应该对错误进行修改

一旦发现了问题,就要解决它。所有的设计小组都有一些非正式的系统来追踪这些问题并解决它们,但是这些非正式系统的性能和可扩展性还有待改进。

2.10.1 什么是待解决的问题

哪些是值得解决的问题

在讨论追踪问题的各种方法之前,首先要明确哪些问题是值得追踪的,这在很大程度上取决于使用的追踪系统。追踪某个问题的代价不应该高过这个问题本身带来的价值。另外,是否需要追踪系统判断问题的类别?或者是否应该先判断什么构成了一个值得追踪的问题,然后再运行合适的追踪系统?后一个问题才是与最终目标紧密联系的:保证设计功能的正确性。

有待解决的问题是指任何能影响设计功能的因素:

1. 在测试平台运行过程中发现的错误显然是值得追踪的问题。
2. 规范文件中出现的模棱两可的情况或不完善的地方也是值得追踪的问题,但是拼写错误不属于此类。

3. 结构上的考虑和折中也属于待解决的问题。
4. 在设计的各个阶段发现的错误，无论是设计本身还是验证环境都应该予以追踪。
5. 如果有新的相关测试方案，也应该归为值得考虑的问题。

如果有值得怀疑的问题，就追踪它

要对所有值得追踪的问题列出清单是不可能的。当问题出现时，判断它是否值得追踪的惟一标准就是它是否会影响最终设计的正确性。如果某个问题不解决就会导致失败的设计，那么它就必须被追踪。当然，并非所有的问题都是平等的，其中一些会直接影响设计的功能，另外一些只是有一些小的负面影响。所以应该对问题的优先级进行分类并依次解决。

可以放弃解决一些问题

一些不太重要的问题，可以不去解决。设计人员可以决定某些特定的问题或缺点在该项目里是可以接受的，可以将其留至产品的下一个版本再改进和解决。这种判断最主要的难点是必须保证放弃这些修改是有意义和有充分理由的。

2.10.2 讨论法 (Grapevine)

问题可以通过口头报告

最简单也是最普遍的问题追踪方法是讨论。在发现问题以后，你可以到硬件设计师工作室中讨论这个问题（假设你不是硬件设计师）。其他感兴趣的设计者也可以加入到讨论中来，一些简单的问题往往可以在这里解决。对于一些较大的问题，可能还需要更多设计者进一步讨论，那么问题的解决实际上就取决于你是否会坚持与硬件设计者进行深入的讨论和交流。

这种方法只在特定情况下才可行

讨论法只在小型的集中式设计组中才能有效地工作，设计人员必须非常接近。如果设计组里有临时或兼职的工程师，或者设计人员在不同的地理位置工作，这个方法就会出问题，因为即时的口头交流并不十分可靠。虽然问题达成了口头的解决方案，但没有人能保证会如期去执行。

不能重复犯同样的错误

这个方法也不能为问题的解决过程保留历史记录。虽然某个问题解决了，但是没有办法将解决问题的过程追溯记录下来。如果解决方案的实施严重推迟，还可能导致同样问题的反复出现。如果提议的解决方案被证明是不恰当的，设计组可能会重头再来，不断重复以前的解决方案。没有历史记录，就不得不频繁地重复以前的操作，设计组将无法从错误中吸取教训。对个人来说，从失败中吸取教训的能力是有限的。

2.10.3 张贴法 (Post-It)

问题可以通过一些小纸条来追踪解决

当设计组越来越大，彼此间的交流不再频繁和有规律时，所使用的追踪方法就是 3M 公司的 Post-It 标签。显而易见，每位设计人员都会在自己的电脑显示器周围粘贴大量提示工作的黄纸条。

如果这些纸条不见了，问题也可能被遗忘

这种改进的方法只是解决了“讨论法”的缺乏责任人的问题：谁贴有这个带有问题的纸条，他就应该对解决问题负责。不过这种责任制是不可靠的，往往很多问题就因为纸条的偶然掉落而被遗忘了。

问题没有优先级之分

在张贴法中，问题没有优先级之分。虽然某个错误对于其他组的设计人员来说可能非常重要，但修改错误的设计人员可能会先处理另一些简单的问题——这样可以尽快减少贴在其显示器上的小纸条。纸条上所有的问题看起来没有区别，没有标出哪些更重要。

仍然会不断重复历史操作

张贴法也存在与讨论法同样的问题：没有历史记录。由于没有记录，问题往往多次重复出现，解决问题的过程也是如此。

2.10.4 过程法 (Procedural)

问题可以通过小组会议来追踪解决

对问题追踪的方法的进一步改进就是程序法。在这种方法里，问题通常由电子邮件信息之类的自由格式文件正式报告出来，一些重点问题可以在小组会议时讨论和解决。

只有最大的问题才被追踪

由于在该方法中涉及到整个设计组的人员，并且会议的细节问题往往都被记录下来，所以它对设计组的交流学习是很有帮助的。但这种方法也会占用小组会议的宝贵时间和精力，因此通常只用于那些特别重要或有争议的问题。那些为数众多的细小问题一般就采用讨论法或张贴法。

2.10.5 计算机化的方法

问题可以通过数据库来追踪解决

对问题追踪系统的再进一步改进就是基于计算机处理的方法。在这种方法里，首先对问题进行深入的分析，未解决问题被重点、多次重复报告。解决问题的任务被正式地分配给某

个或某几个相关的设计者，计算机系统可以自动地将每天或每周的工作报告发送给相关的责任人。

各种不同的解决方案和效果、问题的最终解决方法和过程都被记录下来，这样同样的问题就不会再次发生，而且任何人都可以参照以前的方案来解决或避免发生类似问题。

这种方法比口头或书面报告更难以实现

即使有上述明显的优点，计算机化的方法仍不是很有成效，它最主要的缺点就是不方便使用。讨论法和张贴法在任何情况下都能轻易地使用，设计工程师们面对着紧张的工作日程，如果有以下几项可以选择，你会选哪一个相对简便的方法？

1. 直接面对负责解决问题的人并口头向他报告问题。
2. 在方便贴纸上描述该问题，然后交给上述责任人（如果他不在，则将该纸条贴到他的电脑显示器上）。
3. 将关于该问题的描述输入数据库，然后在计算机上密切关注进展情况。

提交问题所用的时间应该比解决问题所用的时间短

当然你会选择一种最节约时间和精力。如果希望设计小组有效地使用计算机化的问题追踪方法，那么就应该选择一种对设计人员的正常工作影响程度最小的方式，最好是他们的日常工作方式或已在使用的工具的简单扩展。

在笔者曾经做过的一个项目中，使用的问题追踪系统是一种基于X协议的图形化界面。它要花费约15秒的时间才能将整个界面呈现在显示器上，接下来有一系列的菜单选项用来指出问题所属项目、系统、子系统、设备以及功能，还有几个对话框用来描述问题的具体情况。哪怕输入一个最简单的问题都要花费至少三四分钟的时间，而且这个系统还不能从家里以拨号上网方式接入。可以想像这个系统的应用情况.....

使用最广的是基于电子邮件的系统

最成功的计算机化问题追踪系统一般都采用电子邮件界面的方式，通常还带有网页界面来管理任务和报告问题。因为每个设计者都会使用电子邮件，所以电子邮件界面成为了首选，它可以使位于不同地理位置，不同时区的设计者方便地讨论同一个问题。只需要对这些电子邮件信息进行分析、分类，随时跟踪问题的状态和解决方案（通常是对电子邮件正文或标题的特定区域进行简单的设置），就可以有效地实现问题追踪系统。

2.11 数据

数据是最基本的管理工具

管理者都希望看到数据和报告，因为他们没有时间亲自跟踪项目的进展情况，而是根据一些重要的数据来掌握当前的情况。

根据数据随时间的变化寻找趋势

数据一般都以静态的形式出现：“当前的统计值是多少？”“它和最终目标还相差多少？”静态的数值，例如，汽车的里程表可以告诉你已经走了多远，但随时间变化的动态数据能提供最有价值的信息，根据这些数据不但可以知道当前位置，还可以知道速度和方向。

使用历史数据绘制一条基线

与历史数据进行对比时，数据的变化可以显示出工作的改进情况。只有掌握了以前的工作情况，才能更好地了解判断当前的状况。把历史数据绘制成一条基线来判断你的设计水平是很有必要的，在一个每 18 个月生产力就翻一番的制造产业中，设计水平必须跟上这个节奏。

数据可以评估验证的效果

很多数据都可以用来评估功能验证的状态、进度和效率，我们介绍过的代码覆盖就是其中之一。

2.11.1 与代码相关的数据

代码覆盖并非适合所有情况

代码覆盖测试的是源代码被验证的程度，一般它会随着时间的流逝和设计的进展不断增加，最终趋向 100%。

并不是所有的项目都适合用代码覆盖率来评估。代码覆盖率对于独立的小的设计单元（例如，FPGA、ASIC 或可重复利用的元件）是有效的，但却不适用于由单独验证过的模块构成的大型设计项目。验证这些大型设计的目的是为了检查各个模块的接口是否正常工作，而不是验证每个模块的功能，因此没必要运行所有的语句。

测试用例的代码行数可以反映测试效率

执行验证过程所需的代码行数可以有效地反映执行该过程的代价，所以可以用它来比较新的验证语言或方法的效率。如果新方法能减少需要编写的代码行数，那么它也可以降低验证过程的花费。

代码行的比率可以反映设计的复杂性

被验证的设计的代码行数与测试用例的代码行数的比率可以反映设计的复杂程度，这个比率还可用于预测某个新设计的复杂度和验证花费。

代码修改率应该趋近于零

从源码控制系统可以了解到随时间变化的源代码修改程度。在项目的初期，随着新功能的不断加入和版本的变化，代码会以很快的速度变化。在验证的初期阶段，修复错误也会导致代

码的变化。而随着验证过程的进行,错误越来越少,代码的变化也不断减少。图2.20所示的是一个项目的设计过程中的代码变化率,根据图中的数据可以判断代码是否变得越来越稳定,还可以找出设计过程中最不稳定的阶段。



图 2.20 随时间变化的理想代码变化率

2.11.2 与品质相关的数据

品质是主观判断的,但可以通过测试数据来间接反映

与品质相关的数据与功能验证的关系比其他数据与它的关系更加密切。虽然品质是主观值,但它可以由与设计质量相关的数据来反映,这就好比零售服务的质量可以由顾客投诉次数和重复投诉的数量来反映。

功能覆盖可以反映测试用例的完整性

功能覆盖测试的是设计中观测到的输入、输出和内部信号的组合。对这些数据赋予不同的权重,可以得到一个功能覆盖数据,反映设计的功能被执行的程度。如果增加重要数据的权重,它还可以作为功能验证进展情况的一种度量。在项目的初始阶段,覆盖率以很快的速度朝着100%的目标增长,随着项目的进行,由于存在一些难以达到的功能点,它的增长速度明显放慢。

最简单的数据就是已知问题的数目

最容易收集的数据就是未解决的问题的数量,在统计时可以把问题的重要程度作为权重。在使用计算机化的问题追踪系统时,可以很容易地得到这个数据以及它的趋势和变化速率,由此可以判断问题是在不断增加还是在减少并趋向于零?

代码最终将被淘汰

如果你面对的是一个可重复使用的或已存在多年的设计,那么就有必要测量使用过程中出现的错误的数量。这些错误都是在最初的验证计划中没有发现的,如果它们的数量在使用过程中比以前有显著的增加,这意味着它已经被修改过多次或需要重新进行设计,原先的设计应该被淘汰。图 2.21 说明了在一个可重用设计的生存周期内未解决问题的数量的变化情况。

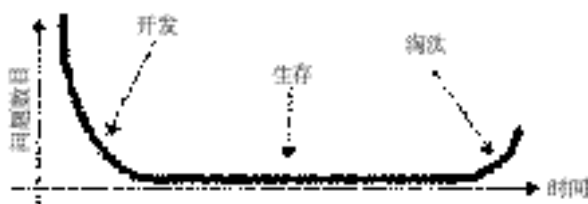


图 2.21 一个设计的生命周期中未解决问题的数目

2.11.3 解释数据

用什么来评估已经完成的工作

管理者在很大程度上依靠数据来判断设计者的工作业绩（作为奖惩的依据），这导致所有设计人员都十分关注数据，也是为什么要慎重选择那些可以客观反映当前情况和所付出的努力的数据的原因。如果评估的是已发现和修改的错误数，你很快会看到这个数目在增加，但你会看到代码的质量在提高吗？错误以前没有被报告吗？如果只要有效地发现并修改错误就能得到管理者的肯定，这会不会导致设计者编写代码时过于马虎和草率呢？

要保证数据与要评估的内容相关

图2.22所示的是两个工程师维护文件清单和当前的版本号。哪个工程师的工作效率更高？左边的工程师使用的版本数较高，这是否说明编写者写的代码有很多需要修正的错误？或者该代码出自一个不断做代码检查的谨慎的编程人员？

alu_e.vhd	1.15	cpuif_e.vhd	1.2
alu_rtl.vhd	1.234	cpuif_rtl.vhd	1.4
decoder_e.vhd	1.12	regfile_e.vhd	1.1
decoder_rtl.vhf	1.155	regfile_rtl.vhf	1.7
dpath_e.vhd	1.7	addr_dec_e.vhd	1.3
dpath_rtl.vhd	1.176	addr_dec_rtl.vhd	1.6

图 2.22 使用版本号作为数据

另一方面，图2.23分别画出了两个工程师的代码变化率。你将如何评价左边工程师的代码品质？依笔者看来，右边的工程师没有很好地运用版本控制系统。

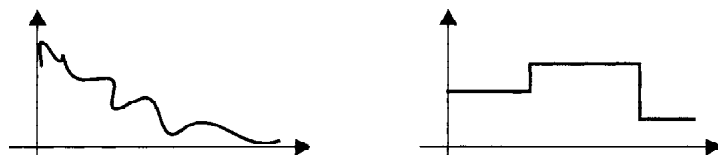


图 2.23 使用代码变化率作为数据

2.12 小结

尽管 Lint 和其他静态代码检查工具可能报告很多的伪错，但它们对于某些错误仍然是最有效的检测工具。

仿真器的好坏取决于被仿真的模型。同时仿真器还提供了许多提高仿真性能的选项，可以支持联合仿真或者混合语言仿真。

基于断言的验证对任何验证方法都是强大的工具，通过它可以很快地发现问题的位置和发生时间。

硬件验证语言由于其对验证任务和覆盖率驱动的随机验证的支持，它对提高设计效率很有帮助。

由代码和功能覆盖数据可以对设计质量进行量化的评估。但要注意的是：不必付出所有代价去达到 100% 的覆盖率，即使达到了预期的覆盖率目标也不能说明设计工作已经完成。

源码控制系统和问题追踪系统可以管理代码并报告错误。

第3章 验证计划

本章将验证计划定义为对功能验证的测试用例以及支持它的测试平台的基本结构的详细说明。在其中将会定义什么是首步成功，怎样验证设计，要编写怎样的测试平台等问题。

你现在面对的设计任务将会为你的公司带来新的市场份额和利润。系统结构设计已经完成，而且确定了一个能够满足功能和成本要求的系统。项目主管们开始根据设计规范编写详细的功能规范文档，这些功能规范文档用于指导实现新产品的 ASIC 或 FPGA 的设计。几组硬件设计师被分配去完成 ASIC 或 FPGA 的硬件实现。在详细的设计规范文档的指导下，他们提出详细的实现计划。到目前为止，似乎这项工程可以满足它的预定产品设计周期。

你正在负责这项设计的验证工作，该产品不仅要求准时完成，而且功能必须正确，公司的信誉正依赖于此。项目经理要求你提出一个详细的验证计划并定义你的工作需求。那么你该如何定义呢？

3.1 验证计划的作用

传统意义上，验证是一个自组织的过程

在传统验证过程中，你的决定将会很简单。事实上，你的职位可能不存在，验证将由硬件设计师按照他的意愿完成，而且要在时间允许的范围内进行。每个人都会双手合十祈祷系统的整合顺利进行，板级设计不需要过多的补线。为了减少在修改系统整合过程中出现的问题而带来的成本，许多器件是由 FPGA 实现的。

有一些工具可以帮助确定什么时间完成

第2章提到的工具可以辅助你的验证过程。代码覆盖、功能覆盖、错误发现率和代码修改率是评判你接近目标程度的标准，但它们就像股票市场指数一样，只能提供当前形势的快照。如果记录了一段时间，可以显示趋势和发展状况，但是并不能用它们来预测将来^①。

3.1.1 详细定义验证

需要工具来帮助确定什么时间能够完成

当前的问题是制订一个时间表。必须尽可能可靠地确定何时能够在所需的可信度下完成验证。只有在明确了解待完成任务的详细要求的情况下，才可能决定需要多少人，花多少时间，甚至正在进行的工作是否有必要去做。这正是验证计划所涉及的。

^① 然而，许多财政和体育分析家以预测或解释一个事实上随机的过程而谋生，在这种情况下，大家都常常出错。

从设计规范入手

在确定验证所采用的计划之前,必须拿到一份待验证设计的规范文档,而且它应该是书面格式的。“口语化”的设计描述说明,例如“除了时钟频率增加为原来的两倍和这些新增加的功能外,与我们先前的设计完全相同”这类的描述是不充分的。设计规范通常包括两个在不同抽象级别上的文档。

- 第一个是结构级的规范,其中详细定义了器件的功能要求。
- 第二个是设计规范,它详细描述系统结构在模块级的具体实现。

当结构级规范文档完成后,就可以开始写验证计划书。当模块级的实现文档完成后,可以用特定的测试用例来扩充完善验证计划书。

规范文档是最好的参考

规范文档是验证和设计的共同出发点,它是最好的参考和准则。随后,当待验证设计产生的实际结果与测试平台期望的结果出现偏差时,设计文档将会判断并决定哪个才是正确的结果。如果需要的话,规范文档应该进一步详细说明来排除这些不确定性。规范文档应当在实现之前给出,实现应当以规范文档为依据。如果说明书依赖于实现或者是实现的一个结果,那么验证就不可能了,因为一旦实现改变,设计规范也将随之改变。

验证计划是验证部分的规范文档

如果没有事先编写好详细的规范文档,今天百万门级的ASIC设计将无从下手。既然验证设计所耗费的精力已经是RTL级设计本身的100%甚至200%,那么为什么它不能有一个自己的规范文档呢?验证计划就是验证部分的规范文档。

3.1.2 对首步成功进行定义

当且仅当首步成功是验证计划的一部分时,它才要被验证

验证计划为整个设计组提供了一个讨论平台,用于定义什么是首步成功。它是一种确保所有基本功能都被适当地验证的机制。如果想看是否取得了首步成功,就应该确定在哪些条件下哪些功能需要测试,还要看期望的结果是怎样的。验证计划中列出了哪些功能是首要的,哪些是可选的。面对时间安排表的压力,在做丢弃某些功能的决定时需要格外留意。与此相反的做法是保留所有可能的功能,那么当提交设计时就会将验证工作突然打断,而某些对市场来说相当重要的功能可能就被错过了。

根据验证计划可以得到一个详细的时间表

验证计划为验证任务划出了一条线,从市场角度看,如果超出这条线就可能会威胁到项目的成功。一旦计划制订完毕,就可以清楚地知道需要有多少个测试用例,还可以知道这些测试

用例的复杂程度及它们之间的相关程度。可以定义一个详细的验证时间表，将任务逐一分配，尽可能让验证并行。一旦 RTL 设计通过了所有的测试用例，而且对覆盖度和出错率也满意的话，就可以提交设计了，在这之前绝对不可以。

项目组要对验证计划负责

让与项目相关的每个人都认识到自己对验证计划负有责任，这是很重要的。RTL 工程师的职责并不只在 RTL 代码的设计，那只是实现最终目的的一种方式而已，他或她的责任是完成一个能够工作的设计。整个项目组都应该对验证计划有所贡献，以保证它是完全并且正确的。

这个过程并不是创新性的

编写验证计划的过程并不新鲜。NASA、FAA 和一些航空航天公司采用这个方法已经有几十年的历史，目的是要确保他们想要实现的可靠性超高的系统满足最终的指标要求。这个过程同时被用在软件设计和硬件设计中。

3.2 验证的层次

验证可以在多个粒度层次上进行

在考虑验证计划时，首要的问题是确定验证任务的粒度层次。一个设计可能由多个层次组成，有些具有物理划分，例如，印制电路板、FPGA、ASIC。还有一些具有逻辑划分，例如，综合后的单元，可重用的元件或子系统。

如图 3.1 所示，验证的每个层次都对一个特定的应用或目的是最适合的。与传统的设计过程相比，在物理层次上的单元级验证结束和系统级验证开始的时候，带有可重用元件的设计的特性发生了转变。可重用设计并没有减少验证的必要性，而是单元与系统之间的边界由物理边界转变为逻辑边界。

在决定不同粒度的层次时要考虑到折中

较小的划分比较容易验证，因为它们提供了更大的可控性和可观测性，建立感兴趣的条件和状态组合，也更容易观察结果是否与期望的相同。在更大的划分下，只有它包含的小划分的组合体才被验证，但以牺牲可控性和可观测性为代价。

在一个给定的粒度层次下进行验证需要稳定的接口

由于验证的实现需要耗费巨大的精力，所以任何待验证的划分都应该有稳定的接口和预期的功能。如果接口不断变动，或者在功能上总是从一种划分变为另一种划分，那么测试平台也需要不断改变而整个验证过程却没有什么进展。一旦确定了将要验证的特定划分，它们的接口和总体功能也应该尽早确定，而且要尽可能不做改动。最理想的情况是，每个被验证的划分有它自己的规范文档，或者至少在规范文档中有它专属的一部分。

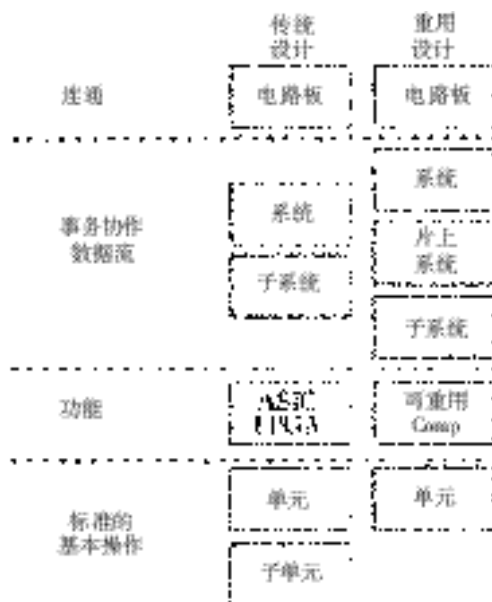


图 3.1 不同验证层次的应用

3.2.1 单元级验证

实现决定了划分的内容

设计单元是逻辑上的划分。它们被用于帮助实现或综合过程，它们可小（例如FIFO或状态机）可大（PCI从接口或DSP数据路径）。由于实现的细化能够突出初始设计中的各个缺陷，设计单元的接口和功能也会随时间而改变。通常没有独立的规范文档来对每一个设计单元进行核实。

对设计单元采用自组织方式验证

由于这些设计单元通常总是处于持续的变化状态，它们最好采取一种自组织的验证过程。设计者自己验证该单元的基本操作，这种验证的目的是为了保证RTL级代码中没有语法错误，它的基本功能也是有效的。并不需要创建一个可回归的测试套件和达到高的代码覆盖率。

它们数量巨大而不能正式地验证

在任何项目中设计单元的数目都是巨大的，因而在该层次上实现验证将会非常浪费时间，每个都需要一个像第5章和第6章中描述的那样的定制验证环境。前面的验证资源将会花超长的时间用于对无数常常变化的接口创建激励发生器和响应监视器，编写众多简单测试平台的工作量并不比编写几个复杂测试平台的工作量小。而且ASIC或FPGA级的验证仍然需要验证这些设计单元的合成。

在大的器件中，可能需要单元级验证

在当今大规模和复杂的 ASIC 和 FPGA 中，很可能无法在 ASIC 或 FPGA 划分的验证中获得必要的功能覆盖，而且不是所有的单元都是等同创建的。对于敏感性和功能复杂度高的单元来说，进行单元级验证从而获得足够的可控性和可观测性，以达到所要求的可信度可能会更有效。最理想的是每个在单元级被验证的功能单元都有它自己的规范文档。

将设计结构化以方便单元级验证

如果设计特别复杂，需要进行一些单元级验证，那么在设计时就应该尽可能考虑到单元级验证的相关性和完整性。要将设计合理划分，使待验证的功能被完整地包含在单元内，而且可以在单个基础上进行验证。验证完成后，可以假设这些功能能够工作在更高层次的验证任务中。如果单元层次的待验证功能需要与其他单元相交互，那么需要在完全包含这些功能的更高层次中进一步验证，以确保在合并后仍能正确实现。

3.2.2 可重用元件的验证

可重用设计独立于任何特定的应用

可重用元件的设计基于独立的指标，它们可以不被修改地用于不同的设计中，它们的可重用性可以被限制于单独一种产品，一个系列的所有产品，或者具有相同功能的所有同类产品。它们的设计和验证都必须独立于任何一个应用。采用断言（参见2.8节）来对可重用元件的输入进行限制和要求是很好的方法，它们有助于确保可重用元件始终按照期望的那样被使用。

验证元件也可以重用

可重用元件通常采用标准化的接口来设计，这些接口是标准片上总线，或者工业标准的外部物理接口等。用于激励和监视这些接口的验证元件自身也可以在多种验证环境中被重复地用于验证不同的可重用元件。验证所耗的精力也可以在多元件应用中被平衡，因而将验证的总投资减到最小。第6章将详细介绍如何对测试平台进行结构化处理，从而促进可重用验证元件的创建和使用。

可重用元件需要一个回归测试套件

可重用元件被用在许多设计中，当对它们进行修改、问题修复或者增强扩展时，必须确保它们向后兼容。这可以由一个回归套件完成，回归套件用于验证元件在修改后的兼容性。除非修改不是功能上的，否则无法采用常规的验证方法来比较修改前后是否相同。由于加入了新功能或者修复了问题，新版本的设计与先前的设计不再等同。

它们需要详尽的功能覆盖

如果潜在的用户对一个元件的性能和可靠性的信任程度低于他们自己能够设计的元件,那么这个元件是不可能被重用的。只有通过详尽、记录完好的验证过程,对元件正确性和健壮性的展示才可能提升用户对产品的信任度。

3.2.3 ASIC 和 FPGA 的验证

物理划分是一个理想的验证级别

ASIC 和 FPGA 都是物理划分的。它们形成了一种天然的验证划分,因为在对系统进行的初始说明和设计规范文档完成后,它们的接口和功能几乎就不太会改变了。

它们可能必须作为系统来处理

由半导体技术发展而引起的不断增大的密度使得将复杂功能集成于单个器件成为可能。从设计和验证角度处理这种复杂度问题,器件通常被设计为一些单独设计和验证的元件的集合,这些元件通常是重用的,但也不必一定如此。在这种情况下,ASIC 被称为片上系统(SoC),它的验证类似于系统级的验证过程(将在下一节详细介绍)。大部分功能验证采用单元级验证。

现在FPGA 的验证需要像ASIC 一样的验证过程

传统上,FPGA 可以通过自组织验证,甚至可以完全缺少验证过程。它们不需要额外的成本就可以重新编程,允许在最后时刻修改功能。但今天百万门级的FPGA,甚至只在50%有效使用的情况下,就可以完成非常复杂的功能,这些功能复杂到在整合后难于验证和调试,因而这些功能必须在综合和实现前从RTL 级代码开始验证。

3.2.4 系统级验证

系统不需要服从物理边界

每个人对系统的定义各不相同。在本书中,系统是由被独立验证过的元件所组成的逻辑划分。因而系统可以由一些可重用的元件组成,包含片上系统ASIC 的一个子集。系统还可以由在几个不同印制电路板上的ASIC 组成,如图3.2所示。

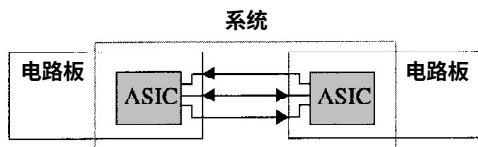


图 3.2 逻辑系统划分

验证的重点是交互

单独的元件在由个人或组来规划和设计的过程中，都要假设它们将与其他元件相互影响。这些由不同人给出的假设是错误的主要来源之一。系统级的验证因而着重考虑各个元件之间的交互，而不是每个元件的功能实现问题，后者在元件级验证中会得到更好的验证。系统验证工程师假设每个元件在功能上都是正确的。

测试用例决定了系统

由于系统是逻辑划分的，它可以由任意个数的元件组成，而不必在意它们的物理位置。使用和验证哪个系统取决于当前感兴趣和有意义的测试用例。为了减少前面的仿真过程，最好采用可能的最小系统来执行指定的测试用例。然而，系统的数量有可能非常巨大，这时候需要定义一组“标准”系统。在多个测试用例中可以采用相同的系统，即使这个系统包含了在某些情况下并不需要的元件。

3.2.5 板级验证

板级模型由电路板设计工具产生

板级验证的主要目标是确定由电路板设计工具所获得的“系统”是否正确。与逻辑系统模型不同，电路板设计的模型必须由电路板输入工具自动生成。在验证电路板设计或其他物理划分时，必须确保当前被验证的就是将被制作的。在输入的设计与待仿真的设计之间必须有直接联系，输入工具自动生成的板级模型可以提供这样的一条联系。逻辑系统模型没有这样的限制：对感兴趣的系统可以手工生成。

电路板上有些元件并不适合数字仿真环境

板级模型最困难的问题是获取所有元件的合适的模型，这是第三方硬件建模器大显身手的地方（参见2.3节）。同样，由电路板设计工具生成的模型涉及到引入近似的问题。例如，在数字仿真环境中如何代表电容？模拟器件、连接器、光耦合器和其他用于板级设计的元件在数字仿真环境下的转换也不容易。

板级寄生效应可以建模

生成的模型可能会包含板级寄生效应的模型，这些寄生效应可能会影响电路板的功能正确性。随着电路板上信号速度的加快，传输线的影响变得严重。ASIC设计时再也不可能不考虑它们最终在电路板上的使用情况了。

3.3 验证策略

对不同粒度的级别采取黑盒或白盒的方法

给出了待验证的功能后,应该确定一种能够执行验证的策略。还需要确定验证将要进行的粒度级别。测试用例可以是黑盒或白盒,根据每个待验证单元的可观测性和对其内部实现的了解程度而定(参见1.6.1节和1.6.2节)。

决定测试用例的抽象级别

还应该决定大部分验证操作将要进行的抽象级别。高抽象级别大多数用在粗粒度的待验证设计中,抽象级别越高,在时序控制、激励和响应方面可做的工作就越少,但生成更多激励和验证长时间的响应会变得更加容易。如果某些测试用例需要进行更具体的控制,那就有必要让它在低一些的抽象级别工作。

应该在周期或器件驱动级验证处理器接口

例如,处理器接口的验证可以在读和写周期级完成。但那要求每个测试用例都对内存映射的寄存器有充分的了解并且知道如何对它们编程。相同的接口还可以在器件驱动级被驱动。测试用例能够通过一系列高层次程序调用完成整个操作,每个操作都由多个对内存映射的寄存器进行读、写的周期组成,但测试用例却可以避免这些实现细节。

3.3.1 验证响应

制定验证响应的计划

怎样进行激励相对比较简单,因为你对它的时序和内容有着完全的控制。而验证响应则比较困难,你必须计划怎样决定预期的响应,如何验证设计提供的响应是所期望的值。6.4节提出了实现输出验证的几种技术。

有些响应在仿真中的验证非常困难

在本书中,推荐使用自检查测试平台(参见5.3节)。尽管对人来说这相当容易,但有时候让测试平台检查一个响应的对错与否是十分困难的。例如,验证一个图形引擎就要检查它的输出图像是否是期望的内容。自检查仿真在验证一幅图像中的单个像素方面是非常有效的,但对于一个填充了红色的圆,人的识别更有效。验证策略必须找到一种方法能自动进行这些验证。

尽早发现错误

最好能让仿真产生一组输出,随后与一组参考输出相比较。仿真的结果可以在仿真器外进一步处理以决定对错与否。然而,最好能够尽早发现问题,当在仿真中检查响应时,能够在离发生错误状态较近的位置找到错误,这样将更容易发现和修复错误。

3.4 从设计规范到功能

确定功能

编写验证计划的第一步是确定要被验证的功能。从规范文档中可以列举出它所描述而且要被验证的所有功能。其他的项目组成员，尤其是系统结构和RTL设计师，进一步补充待验证的功能，对不熟悉设计目的和特点的人，这些新增的功能在设计规范中可能不是非常明显。在选择某个特定的实现后，其他的某些功能可能会变得重要起来。在“The Art of Verification”一书中^①，Haque、Michelson和Khan提出了一种系统的方法来提取有意义和相关的功能，首先观察接口，然后是功能，最后是由选定结构确定的边界情况。

列举基于接口的功能

对于待验证设计的每个接口，列举出必须验证的每个功能。基于接口的功能可以由以下问题获得：

- 必须采用哪些事务激励？
- 值的范围是多少？
- 事务的次序是怎样的？
- 相关的事务比例是多少？
- 设计应该能承受什么样的协议违返？
- 这个接口与其他接口之间，或与内部设计结构之间有什么相关的相互影响？
- 一个接口的事务是否需要与其他接口同步？

通用异步收发器的一个基于接口的功能列表的子集如例 3.1 所示。

例3.1 通用异步收发器（UART）设计中的一些基于接口的功能

1. 当 UART 可以接收 CPU 接口传来的新字的时候，允许发送引脚（CTS）必须置位。
2. 当一个接收到的字准备好被 CPU 接口读取的时候，数据中断准备引脚（DTR）必须置位。
3. 除了地址 0-4 外的其他地址都有读周期和写周期。
4. 在地址空间支持的范围内可连续进行读 / 读、读 / 写、写 / 写、写 / 读 周期。
5. 配置寄存器中的所有位都是可读、可写、非易失性的。

确定基于用途的功能

根据设计中的主要数据通路^②，列举出每个必须验证的变换和选择。基于用途的功能可以由以下问题获得：

- 所有有关的配置是什么？

① Faisal Haque, Jon Michelson and Khizar Khan, “The Art of Verification with VERA”, <http://www.verificationcentral.com>.

② 按照结构规范文档中规定的，而不是实现中所指的。

- 可能发生的数据变换有哪些？
- 变换的顺序是什么？
- 触发变换的敏感数据值是什么？
- 影响每个变换的敏感值是什么？
- 变换后的数据在哪里终止？
- 数据顺序是怎样被影响的？
- 存在着什么错误检测机制，它们是怎样触发的？
- 错误机制怎样报告错误？
- 对错误的数据会发生什么？

UART 中一个基于用途的功能列表的子集如例 3.2 所示。

例3.2 UART设计中一些基于用途的功能

1. 数据的发送和接收都以串行方式，最低有效位最先。
2. 数据比特的发送顺序与写入顺序相同。
3. 数据比特的读取顺序与接收顺序相同。
4. 根据配置模式生成奇偶校验。
5. 根据配置模式检查奇偶校验。

列出基于结构的功能

最后，在对设计的结构有了详细了解的基础上，给设计增加压力，将其推向极限的条件。

基于结构的功能可以由以下问题获得：

- 可以使一个缓冲器发生溢出或下溢吗？如果可以，会发生什么现象？
- 哪里是资源的瓶颈？
- 对这些资源的多个请求可能同时发生吗？
- 一个变换的路径是否会影响、阻止或阻塞其他的变换？

UART 中一个基于结构的功能列表的子集如例 3.3 所示。

例3.3 UART设计中一些基于结构的功能

1. 在接收缓冲已满的情况下再多接收一个字节将导致这一字节数据的丢失。
2. 允许发送（CTS）信号能反映发送缓冲的状态（如果不满，将置位）。
3. 数据的接收和发送采取全双工方式。

标注各个功能

应该对每个功能进行标注，并加上简短的描述，描述部分要包括需要验证的条件和期望的结果，而不是它是如何实现的。每个功能都应该与规范文档中详细描述它的那个部分或段落相

对应，规范文档最好也能与验证计划中的功能列表相对应。为每个功能确定适当的验证级别，当出现问题时，功能标签应该以错误信息形式给出，把功能标签包含到错误信息中将有助于确定哪部分可能出错和最终评定是否真正出错。

为功能确定适当的验证级别

在列举功能时，要注意将它们包含在验证计划的适当验证级别中。有些功能适合在元件（单元、可重用或 ASIC）级验证，而有些必须在系统级验证。多数情况下，验证设计中的一个关键功能或模块总是与大量功能相关。如果实现那个功能或包含那个模块的单元没有被单独验证，就需要重新考虑你的验证方法了，这可能暗示着为达到必要的可信度这个单元需要被单独验证。

3.4.1 元件级功能

元件级功能完全包含在待验证单元里

元件可以是一个单元，一个可重用元件或一个 ASIC，元件级功能完全包含在待验证单元里。它们不涉及与其他元件的系统级交互，不需要将元件合成到一个更高层次的系统就可以验证它们的正确性。

大多数的功能都是元件级功能，当元件被用于系统级验证时，这些功能都被假定为正确的。

3.4.2 系统级功能

将系统级功能最小化

系统可以是一个 ASIC 的子集、不同电路板上的几个 ASIC、整个电路板或整个产品。由于系统级仿真的规模大、仿真时间长，所以最小化系统级的验证功能是十分必要的。每当要将某个功能作为系统级功能时，一定要考虑是否可以把它作为元件级功能进行验证。例如，图 3 的设计中，ASIC MX 可以在软件控制下从 ASIC ID0 和 ID1 中选择数据，这个开关功能是系统级功能吗？答案是否定的。开关功能完全包含在 MX 中，因而是元件级功能。

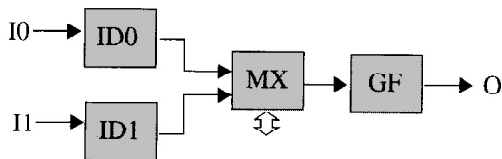


图 3.3 系统结构的例子

系统级功能包括互连、流控制和互操作性

系统级功能通常仅限于互连、流控制和互操作性。例如，由输入端到输出端的连接是系统级功能。在验证互连关系时，有必要将ID0路切换为ID1路。这种切换操作不是验证的主要目标，因此可以假设为切换开关能够工作。

另一个系统测试用例要验证的内容是：在MX ASIC的输入FIFO已满的情况下，会给ID0和ID1带来压力，从而阻止数据的流动直至拥堵解除。

3.4.3 待查找的错误类型

假设设计工具不会引入功能错误

在列举待验证的功能时，对于可能出现并被发现的错误，应有一个隐含的假设。功能验证应该集中于找到设计中的功能错误，而保证设计工具没有错误并不是功能验证的任务。功能仿真确保设计以指定的形式实现，而没有设计者引入的错误或问题。例如，对门级网表运行所有的功能测试平台仅能验证综合工具是正常工作的。形式验证和静态时序分析是实现这个任务的更好的工具。

使用不同的输入工具会产生不同的错误类型

使用不同的输入工具会产生不同的错误类型。使用图形输入工具时，常会出现连接错误，如总线中比特顺序颠倒，或特定位连接错误等。在RTL编码和逻辑综合环境中，不容易发生这种错误。如果连接了一条总线，要么所有比特都工作，要么都不工作。Linting工具能检测一些连接问题，如对一条连线的多驱动，或输出线未连接等，而且它是解决这类问题的较好方法。

查找功能错误

在基于综合的设计流程中，常见的错误包括极性错误、协议冲突或计算错误等。在图形输入工具中被证明很有用的激励类型，例如，walking ones测试和walking zeros测试，在RTL设计验证过程中就不那么有用了，一组交替的0和1，例如，“0xAAAA”和“0x5555”通常就足够了。

在数据流中使用签名也是一种常用的功能错误检查技巧。签名可以是用来检测丢失或重复数据项的序列号，也可以是对数据项的源或目的地进行编码。例如，在写周期中与地址相关的数据可以包括地址的一部分以及产生这个写周期的总线控制器的标识。6.4.2节将详细介绍怎样使用签名来验证某一类设计。

3.4.4 优先级

给功能分配优先级

并不是所有功能都是平等的。只要列举功能，它们就必须按优先的次序加以区分。其中一些功能对于设计功能的正确性或满足市场要求来说是“必须拥有”的，是成功的第一步。只有

它们正常工作以后设计才能被提交。完成这些功能的验证开启了成功完成整个项目的大门，而验证这些功能的测试平台是最关键的部分。“必须拥有”的功能需要对所有可能的配置和使用选项进行彻底的验证。

重要的功能越少，所要的关注越少

“应该拥有”的功能对于商业上成功的设计来说是第二位的。它们也许只是简单地提供功能上的扩展或与其他竞争产品的差别。它们的主要验证目标是验证基本功能的正确操作，在时间和资源允许的条件下对这些功能进行更详细的验证。如果时间进度压力很大，迫使资源都被分配去验证更为重要的功能，那么这些功能的验证会被取消。

一些功能只在时间允许时才被验证

“最好拥有”的功能是可选的。只有在时间允许的条件下，才会用最简单的方式验证它们。按照现实的设计进度表，这些功能永远不会被验证！

如果迫于进度压力需要取消计划中的一些验证项目，那么对待验证的功能划分优先级可以使项目经理做出明智的决定。削减验证任务可以先从不重要的功能开始。如果项目完成的日期对进度提出了更高的要求，使我们要放弃一些“必须拥有”功能的验证，那要保持清醒的头脑来做这些决定，因为这些功能定义的优先级和最初的市场目标一样重要。削减对“必须拥有”的功能的验证需要小心认真地对项目的市场目标进行重新评估。

3.4.5 可验证的设计

确认那些难以验证的功能

在验证计划的这一步，我们要确认那些难以验证的功能。它们之所以难以验证，是因为所选择的设计划分缺乏对这些功能的可控制能力或可观察能力。比如验证一个嵌入式的64比特计数器，它计数到最大值后会重回到零，处理算法必须要在计数器的回零时正确工作。验证的难度可能会来自于对验证粒度的错误选择。在这种情况下，我们应该选择包含验证功能的更小划分。验证难度也可能来自于实现结构的选择或设计本身的人为产物。如果无法使用更小的划分，或者划分不能有助于这些功能的验证，那就必须使用灰盒法或白盒法。

修改设计来帮助验证

将验证计划提前的好处是你仍有机会修改设计的实现。如果一些功能被证明在目前设计的结构和功能集里是难以验证的，就可以修改设计，增加一些附加的功能来辅助验证。硬件工程师无疑会抱怨这些实际并不需要增加的功能，但他们还能有什么选择呢？这些功能在系统集成的实验里被证明总是有用的。

提供状态预加载功能

如果设计中包含很长的计数器或复位后成百上千个周期后才能达到的状态条件,那么就要确保它们可以通过内存映射的寄存器预先加载任意值。理想情况下,可以通过相同的寄存器组回读它们的值。在前面的例子中,设计的地址空间中一系列的八字节可以通过预加载来分配,并回读 64 比特计数器的计数值。

提供数据通道旁通的路径

如果没有对所有操作数详细控制,那么很长的数据通道的正确实现也会难以验证。例如,语音合成器是一个简单的数字信号处理设计,它有对随机噪声整形的数据通道^①。虽然可以控制完整的系数并应用到数据样本上以形成特定的声音,但无法控制一个关键元素:最先的输入数据。那是一个随机数。为了正确验证这个数据通道的操作,需要控制它的初始输入值。

如图 3.4 所示,该设计应该包括一种机制,这种机制可以对数据通道的输入使用可编程的常数值而不是随机数。相反地,你应该能够回读随机数发生器的输出以确保它确实生成了随机数。



图 3.4 可验证的语音合成器的数据通道

小测验：为什么回读的点要选择正常操作使用的随机数发生器和可编程静态值之后的多路选择器上，而不是随机数发生器的输出上^②？

提供采样点

如果可观察能力而不是可控制能力是个难点,可以通过内存映射的寄存器加入一些可读出的采样点,这样可以方便一些功能的验证。如果分配给设计的地址空间很宝贵,这些采样点可以用多路选择器共享同一地址空间,用第二个地址来选择当前的采样点。

提供错误注入机制

如果设计包含错误和异常检测机制,你可能想提供一种方法强制发生错误检测或异常检测。例如,在验证中断屏蔽能力时,如果要强制设计进入每一个异常条件,那么验证是十分费时的。如果一个简单的寄存器写操作可以手工地触发相同的中断,那么验证起来就容易多了。

- ① 它是用来生成辅音的,如 sh 的发音。然后它与基于整形的频率合起来生成元音,如 a 的发音。这样就可以产生可以理解语音。
- ② 这样可以验证:在数据通道处于正常操作模式时,多路选择器是功能正确的且输入值确实是来自于随机数发生器。

当然，异常条件对中断的触发仍然需要验证。要仔细地考虑把错误注入包含进设计的决定。如果这仅仅是为了硬件的验证，那么可能不会出现在给软件工程师的说明书里。当一个器件的驱动程序写入一个它认为无害的值时，这个功能可能会被偶然地打开。

3.5 直接的测试平台方法

有了直接的测试平台，每个功能可以使用相应的测试平台来验证。激励可根据要验证的功能手工加入。所得到的响应与功能没有正确实现时可能出现的响应进行比较。

这种方法适用于测试用例数量较少的情况

在开始编写直接的测试平台之前，需要考虑到它会缺乏可扩展性。如果测试用例的总数在几百件以内，那么这种方法是可行并可管理的。但随着测试用例数目的增加，测试平台的数目也会增加。一个有上千个测试用例的项目如果使用直接的测试平台会需要超过一年的时间来完成。对于一个有更大数目测试用例的项目，有必要使用一些测试平台自动化方法，以便在可接受的时间里完成验证。目前，测试平台自动化最好的方法是覆盖率驱动的基于随机的方法（参见 3.6 节）。

3.5.1 测试用例的分组

把有相似验证需求的功能分为一组

功能分组是很自然的。一些功能需要相似的配置、粒度或验证策略来进行验证。为了最大化生产效率，这些功能应该被分到同一组，并分配给同一位验证工程师。例如，所有和 CPU 接口有关的功能应该被分到一起。再举个例子，对串口波特率、数据比特数和奇偶生成的验证应该分到同一组。每一组功能的验证形成一个测试用例。

功能清单中的交叉引用

每个测试用例应该有一个标签和对目标的一小段描述。描述应该包括测试用例中已验证功能的清单。功能的清单应该链接到验证这个功能的测试用例。如果一个功能没有指向一个测试用例的交叉引用，那么这个功能就没有验证。

定义从属关系

对测试用例的描述应该包括被认为可操作并功能正确的功能的清单。依靠它们的从属关系，可以决定测试用例编写的顺序，并确认在测试平台开发中是否有并行的机会。

指定测试用例的激励

必须描述测试用例激励的顺序和特性。例如，要描述必须进行的各种操作或总线周期。用随机数据或随机事务来填充所有不相关数据或背景数据是个好方法。

指定可接受的准则

除了期望的响应，测试规范还必须说明如何判定响应是正确的。这包括期望的值、时序和协议。例如，路由包处理器的输出的目标地址如果和它显示的输出端口是匹配的，那就可以判定是正确的。或者可以用一些更严格的判定方法，例如，不同来源的包以一定的顺序排列并以一定分布交织排列。

指定要查找的错误

一种更直接的描述可接受准则的方法是描述要查找什么样的错误。例如，确认一个包有正确的 CRC 校验值。另一个例子是描述不能同时发生的事件，如 FIFO 中 full 标志和 empty 标志的置位。直接描述要查找的错误可以让一个不太熟悉设计的验证工程师实现高度可靠的测试平台。

注入错误以确定它们被检测到

永远不要相信一个不生成错误信息的测试平台。每个测试用例应该包括一些错误注入机制来确保测试平台可以发现并且报告错误。缺乏错误信息可能会成为测试用例的一个失败条件。比如，一个用于验证串口奇偶位生成的测试用例应该能故意错误地配置奇偶位，以确保测试平台能够检测到一个错误的奇偶位。当然，测试平台不能一发现错误信息就退出仿真。

3.5.2 从测试用例到测试平台

测试用例可以自然地分组

和功能一样，测试用例也可以自然地分组。它们需要设计使用相似的配置，对激励和响应使用相同的抽象层次，生成相似的激励，使用相似的策略来判定响应的正确性，或验证密切相关的功能。例如，验证串口正确传输数据的测试用例可以和验证串口配置控制的测试用例分为一组。它们都需要相似的激励（发送各种不同的数据字），它们以相似的方式验证输出的正确性（数据的值要相同，没有奇偶校验错误）。

把测试用例分组到测试平台

然后每一组测试用例被划分到不同的测试平台。本书所使用的是一种流行的划分方法，即一个测试用例一个测试平台。把几个一组的测试用例合并在一个单独的测试平台，减小测试平台的数目，这样可以最小化 Verilog 的编译时间，或最小化一个大的门级网表和一个大的标准延时文件（SDF）的反向标注时间。

测试用例和测试平台交叉引用

每个测试平台应该被惟一地标注和确认。这个标识符可以被用做文件名，测试平台的顶层代码可以据此实现。对于每个测试平台，要列举出它所实现的测试用例清单。在测试用例清单

里交叉引用测试平台。测试用例的描述应该包含实现它的测试平台的名字。如果没有引用任何测试平台，那么说明这个测试用例还没有被任何测试平台使用。

为每组测试用例分配一名工程师

不管测试用例是如何分组到测试平台的，每一组测试用例应该分配给一名验证工程师。同一组测试用例有相似的实现需求。它们可以在同组中前一个测试用例的实现基础上完成。第一个测试平台要花费最长的时间来完成。但是随着负责每组测试用例的工程的经验增长和对基础结构验证的调试，很多工作可以被重复使用。在接下来的测试平台中，只要剪切复制就可以完成工作。每个测试平台被分配的人员名字应该被记录在验证计划中。那个人根据验证计划的规范文档负责完成测试平台。

3.5.3 验证测试平台

如何验证测试平台是按照验证计划实现的

验证工作和编写测试平台的目的是为了验证设计能够满足设计规范的要求。如果验证计划是验证工作的说明，如何验证测试平台是按照它们的说明实现的？如何防止由于人为的错误而错过了相当一部分测试用例？测试平台经常包含一些临时代码结构来跳过测试平台的很多部分以加速关键部分的调试。如何确定测试平台的哪些部分已经被去掉了，测试平台实现了它应该包括的所有测试用例吗？

通过同伴的检查来验证测试平台

正如 1.4 节所述，一种验证人为的变换（在这里指根据规范编写成测试平台）的方法是提供冗余。一旦测试平台完成，它就要被其他验证工程师重新检查以确保测试平台实现了它应该包含的所有测试用例。更详细的信息可以参见 2.1.5 节。仿真输出的日志也应该被重新检查以确认测试平台是按照规范执行的。为了达到这样的效果，测试平台应该产生规则的报告，表明什么样的激励正在生成，什么样的错误和响应被检查到了。输出日志应该最终总结说明被执行过的测试用例。

使用功能覆盖

另外一种冗余的方式是功能覆盖测量。通过功能覆盖模型，可以指定一个直接的测试用例要完成的事，可以明确地了解测试用例执行的情况。在直接的测试平台运行以后，功能覆盖机制应该 100% 地满足目标。由于激励是手工编码的，它是确定的并 100% 地覆盖了相关的和感兴趣的值。比如，一个测试用例要使某一特定的配置寄存器重复所有可能的值，这个测试用例可以通过记录所有写到配置寄存器指定地址的值来覆盖。

3.5.4 测量进度

用测试用例的完成来测量进度

在直接的测试平台方法中,进度是用一个简单的表格来测量的。一方面,它列举了所有的测试用例。另一方面,通过跟踪每个测试用例的生命周期来确定测试用例的当前状态:分配、编码、运行、传递、检查。图3.5显示了一个直接的测试平台方法的进度。最初,由于验证的基础结构仍在开发之中并且设计也在调试之中,因此验证基本没有进展。一旦第一个测试用例成功地完成,随着未发现的错误越来越少,可重用的基础结构代码越来越多,验证的进度将会加速。这种加速也许并不会加速测试用例完成的速度,因为测试用例的实现将会变得越来越复杂。

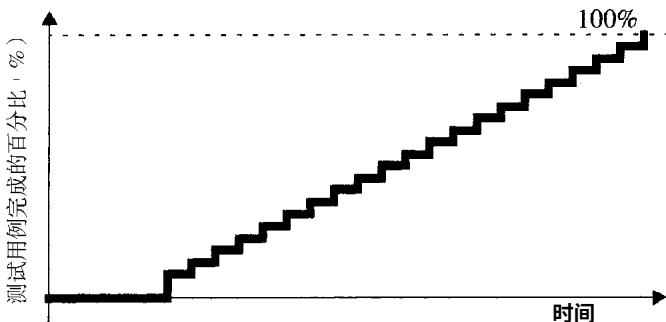


图 3.5 直接的测试平台方法的进度

什么时候能够完成

所有测试用例的完成并不意味着验证任务已经结束了。代码覆盖机制可以指出原来的测试用例并不像所想的那样周全,必须加入其他测试用例来把代码覆盖率提高到可接受的水平。实际上,只有在提交设计并对“必须拥有”的功能有足够信心的时候,才可以认为验证“完成”。

3.6 覆盖率驱动的可约束的随机验证方法

HVL 生产周期(参见 2.7 节)依赖于受约束的随机验证。可以使用 HVL 实现前一节所描述的直接的测试方法。高层次的编程语言结构有助于测试用例的实现。实际上, HVL 对功能覆盖的支持更少,由于这个原因才设计了受约束的随机生成。然而,这可能只会增加测试用例完成曲线的斜率(如图 3.5 所示)。要改变曲线本身需要改变验证的方法。

随机验证仍然要提供有效的激励

随机验证并不意味着向设计的输入信号中随机地加入 0 和 1。这样并不代表设计的正确使用,这样什么也没有验证。在随机验证时,设计输入的是有效的单个操作,如一个读周期或一个以太网数据包。这些操作的顺序和时序以及输送的数据内容是随机的。通过附加的约束,可以控制随机的测试平台朝特定功能的验证发展。

3.6.1 测量进度

有太多的测试用例

今天数百万门的专用集成电路包含着数百个要验证的功能,每个功能有数百种不同的数值组合。假设现在有一个没有错误的设计和一个高效的工程师团队,他们可以在3天里编写并调试一个自检的测试平台,那么一个有10位验证工程师的团队(按照今天的标准,这样大的数目很少见)将需要七个月以上的时间才能完成500个测试用例。测试用例的数目是不能减少的。投入更多的工程师可以进一步减小验证时间。惟一减少验证时间的办法是在更少的时间里写更多的测试用例。换句话说,用更少的代码来实现相同的功能。

测试用例的验证内容要比目标功能多

虽然在验证一个测试用例时,每个测试平台分别考虑目标功能,但是可以给设计输入激励验证一个功能的同时验证其他功能。由于在直接的验证方法中,进度是通过功能和测试平台联合起来跟踪的,对于那些没有在测试平台里明确编写和验证的功能如何跟踪进度呢?

测量功能覆盖而不是功能

解决方法是根据功能覆盖点来测量进度,这些功能覆盖点可以确定一个功能是否被验证过。这样,验证的目标变为填满设计的功能覆盖模型而不是编写一系列的测试集。你可以用很大的直接的测试平台来填满这个覆盖模型,或者可以让一个随机的测试平台生成测试集替你验证功能。图3.6显示了用带有随机测试平台的覆盖驱动方法进行测量的进度和传统的直接方法测量的进度。前者的测试平台的初始开发时间较长,但长时间运行时的功能覆盖率更高。我们所承诺的最终产能的提高不能从这幅定性的图上测量:因为它高度取决于这种方法的约束,以及你编写易于约束的随机发生器的经验(参见6.6节)。

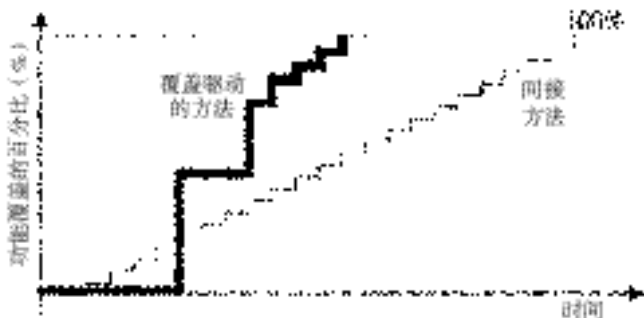


图3.6 覆盖驱动的测试平台方法的进度

你会有更多的信心

直接的测试用例只能发现你想要查找的错误,而随机仿真可以生成一些在写验证计划时未曾想过的条件。它们可以生成意想不到的条件。它们也可以减少验证工程师在编写直接的测

试平台时引入的偏差。代替生成很容易编码的输入序列，它们生成更符合实际的激励。由于设计会在相当多的条件下进行验证（在同样的时间里相对于直接的方法来说），设计整体的质量会更高。

这个方法需要有约定

使用约束驱动的方法需要约定。在进度压力下，很容易回去写直接的测试集。这种方法的一个关键部分是需要对测试平台和设计进行仿真，以弄清楚已经完成了多少功能覆盖。如果RTL模型没有准时完成（它往往不能准时完成），要怎样调试自检验的随机测试平台呢？怎样知道验证团队正在向它的功能覆盖目标前进呢？最简单的解决方法是开始把测试用例编写成直接的伪随机的测试平台，毫无疑问它可以填充功能覆盖点。那会使你又回到楼梯曲线。更好的方法是尽可能早地交付RTL、开始仿真并使用行为级模型。关于行为模型的更详细的信息可以参见7.1节。

3.6.2 从功能到功能覆盖

从功能覆盖开始

在覆盖驱动的方法中，功能覆盖测量用于确认哪些测试用例被执行了，而不是直接编写那些测试用例。因此，实现功能覆盖模型并从一开始就收集功能覆盖率是很重要的。功能覆盖不像代码覆盖，后者常加在验证过程的末尾，用来测量代码执行得有多彻底，并确认那些没有被执行的代码。功能覆盖从项目的一开始就用来记录哪些测试用例和条件被随机发生器自动生成了。如果没有把功能覆盖和随机环境联合使用，那恐怕仅是在用随机激励做直接的测试用例。

测量数据指示的征兆

每种功能都会在输入数据流、设计的配置或必定要经过的设计内部状态中体现出一种特性或征兆。功能覆盖必须确认并记录那些特性和征兆。例3.4显示对功能覆盖项的描述，这些功能覆盖项用于确认例3.1中基于接口的功能已被验证过了。

例3.4 UART设计的基于接口功能的功能覆盖

1. Clear-to-Send(CTS)引脚的电平。
2. Data-Ready(DTR)引脚的电平。
3. 地址交叉的CPU周期类型。
4. CPU周期类型的变化。
5. 地址交叉、数据交叉的CPU周期类型。

定义目标

只有明确地定义了目标，功能覆盖工具才能测量进度。它也会使功能覆盖分析更容易。进度会以一个不变的目标进行测量。如果每次分析功能覆盖报告时目标被智能地定义，那么这些

目标属于人为的错误。因为你会下意识地把进度与迫近的截止日期进行比较,因此在项目快结束时有一种要减小验证缺口的重要性的趋向。例 3.5 描述了例 3.4 确认的功能覆盖点的功能覆盖目标。不同的功能可以使用相同的覆盖点,但意味着更宽的目标。

例3.5 UART设计的基于接口功能的功能覆盖目标

1. 至少观察到一个 0 和 1。
2. 至少观察到一个 0 和 1。
3. 对地址大于 4 的地址至少进行一个读周期和写周期。
4. 所有读写周期的组合。
5. 地址与配置寄存器相同,寄存器的每个比特为 0 或 1,至少进行一个读周期和写周期。

理解目标的复杂性

不要让你的目标比需要的更准确。如果为了实现目标需要设定的值越多,你的工作就越多。由于交叉覆盖,要设定的值的数目会成指数地增长。例如,要想覆盖 2 比特地址的所有可能值来进行读操作和写操作是不现实的:那将有 80 亿个值。对等价的值定义值的集合或者对值进行组合以减少满足目标所需的样本数。功能覆盖工具对于进行覆盖测量的值的集合总数有限制。例如,Specman Elite 仅在目标设定值最多不超过 16 个时才会提供功能覆盖的覆盖百分比^①。

询问、减少与通知

收集大量的功能覆盖数据是很容易的。但是掌握的功能覆盖数据越多,分析结果就越困难。总是需要询问适当的功能覆盖点。如果打算忽略一些覆盖报告或不再关注以后的报告,就不该收集它。数据和信息之间有本质的区别。任意的功能覆盖点仅提供要分析的数据,而仔细选择的功能覆盖点和精心定义的目标可以马上提供有意义的信息。例如,如果要验证一个 FIFO 的操作范围,那么读写指针的值就是一种数据。而测量读写指针的差值可以指出是空状态、满状态或者二者都不是,这会更有意义。如果指针的差值超过了一个关键的指针区域(如溢出点),那么后一种功能覆盖点会提供更多的相关信息。

功能覆盖的定义是一种还在发展的艺术

为验证计划开发好的功能覆盖模型是不容易的。本节概要讨论了必要的步骤。功能覆盖模型是一个可以并且应该被发展的明确定义的科学话题,需要一整本书来讨论它。

3.6.3 从功能到测试平台

确定如何判断正确性

注意,上面所描述的功能覆盖点并没有涉及到结果的正确性。正确性由测试平台的自检验部分负责。对于必须要验证的功能,要决定如何判断结果的正确性。这个过程与在验证功能时

^① 那个数是可配置的。这是一个默认值。

判断直接的测试用例的期望响应相似。区别是你并不知道触发功能的激励的时序和顺序。有多种手段可以检测到错误：随机的测试平台正常工作下的错误，直接用自检验结构比较输出数据和期望的数据，或把设计本身声明为一个白盒。错误检测机制的清单可以成为自检查随机测试平台的详细说明。例 3.6 显示了可以确认例 3.1 的功能正确性的错误检测机制。

例3.6 UART设计的基于接口功能的错误检测

1. 在写入下一个要发送的数据之前，数据源会等待 Clear-to-Send(CTS)引脚的信号置位。如果功能不正常，就不会发送数据。
2. 数据接收器在读进下一个要接收的数据之前会等待 Data-Terminal-Ready(DTR)引脚的信号置位。如果没有信号置位，就收不到数据。
3. 由第 4 项包含。
4. 验证读周期所返回的值是先前所写的值，验证比特和复位值的写入能力，验证寄存器的容量和内容。
5. 由第 4 项包含。

确定终止机制

终止一个直接的测试用例是很容易的：一旦完成了验证目标功能所需的激励，就可以从仿真器中退出。但是随机的测试平台并不是验证单一功能的。怎样知道什么时候该停止呢？要准备一些停止机制，这些机制可以通过在随机的测试平台上加入约束或者在仿真运行的开始调用一个简单的过程来触发或关闭。任何一个停止机制，一旦被触发就会有序地终止仿真。

有一些流行的停止机制。看门狗定时器在防止仿真死锁时十分有用：每隔一段时间它就必须被复位一次，否则仿真就会被终止。定时器可以防止仿真跑飞：在一段预先定义好的时间之后它会终止仿真。空闲检测器会在检测到输出接口处于空闲状态一段时间以后终止仿真。一个简单的数据项计数器会在发送或接收到一定数目的数据项以后终止仿真。如果达到了覆盖目标或覆盖率没有明显的增加，功能覆盖反馈也可以终止仿真。

可以运行很长时间，或者运行很多次

可以用两种策略生成很多随机数据：可以运行一个随机数据源很长时间，或者可以在短时间里运行一个随机数据源很多次，每次用不同的种子。如果随机数据源确实是随机的，并且所选择的种子不会重复先前的序列，那么所得到的随机数据的质量是相同的。但是这两种策略的仿真效果是非常不同的。

准备运行很多次短时间的仿真

如果一个很长的仿真快结束的时候出现了错误，那么再现它是令人讨厌的。进一步说，由于单次的仿真运行一般只使用单个的器件配置，验证不同配置的机会会更少。你可能发现器件处在状态空间的一个特定的边界，并且总是呆在那里。在那种条件下运用更多的激励不可能增加功能覆盖。

使用很多次短时间运行的策略,可以更快地再现错误,运行更多的配置并很快地穿过状态空间。由于HVL独立于Verilog和VHDL,没有必要重新编译设计去运行不同的仿真。如果设计在复位后需要很多的仿真周期才能达到一定的状态,可以考虑用3.4.5节中描述的强制状态机制。

3.6.4 从功能到发生器

确定激励的需求

为了能够满足覆盖目标,随机发生器必须以必要的特性和必要的时序生成必要的数据。用随机的内容生成单个路由包或指令是很容易的,但是当你需要生成不同长度的路由包、许多相同长度的连续路由包、直接通过的指令序列、嵌套的循环结构、非法或错误的数据、或者穿过多个随机流的同步数据时,那么上面的简单的随机生成方法是不够的。

随机发生器能够验证所需的功能并非偶然。它的结构必须能产生所需的数据序列。例3.7显示了验证例3.1中的功能的随机发生器必须满足的要求。

例3.7 UART设计的基于接口功能的发生器的需求

1. 生成要发送的数据流。
2. 生成要接收的数据流。
3. 生成读写周期。
4. 第3项已包含。
5. 第3项已包含。

使用约束会比使用多个种子更好

在多次随机仿真以后,一些功能覆盖点仍然没有被填充该怎么办呢?可以用其他的种子运行更多的仿真,或者可以在测试平台里加入约束来提高对剩下的功能覆盖点的覆盖的可能性(希望是100%)。后一种方法,虽然需要进行更多的工作,但可能是效率更高的方法,特别是在前面数百次仿真都没能产生填充那些功能覆盖点的必要输入的情况下。功能覆盖测量的空白很可能也是随机发生器或验证环境存在功能问题的症状,很可能一个未删除的约束使随机发生器不能产生填充功能覆盖点的输入序列。

确定可约束的维数

当构造随机发生器时,有必要考虑可用的约束机制。通常不同的随机流是通过物理上改变随机发生器的代码来产生的。如图3.7所示,改变随机发生器的代码有效地为每个测试平台建立了不同的随机发生器。

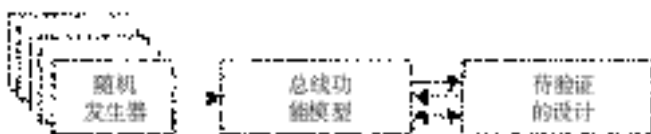


图 3.7 不同的随机发生器

为了减少重复的代码量以及减少填充额外的功能覆盖点写的新代码（从而提高效率），最好设计一个可以很容易从外部约束的随机发生器，如图3.8所示。编写可以很容易从外部约束的随机发生器并不是偶然发生的。6.6节描述了如何编写这样的发生器。例3.8显示了要验证例3.1中的功能而必须加在发生器上的约束机制。

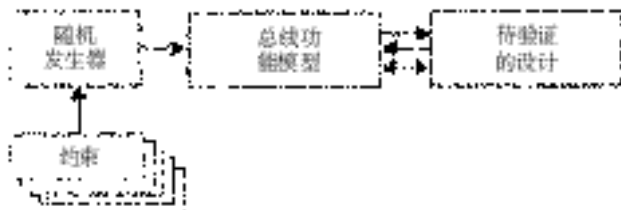


图 3.8 约束单个随机发生器

例3.8 UART设计的基于接口功能的约束需求

1. 无约束。
2. 无约束。
3. 必须能够约束地址。
4. 必须能够约束周期序列中周期的类型。
5. 必须能够约束地址和数据。

随机地产生器件和测试平台的配置

很容易想到在仿真中始终使用随机生成的数据，但也可能简单地在仿真开始时随机产生只用一次的数据。例如，可配置或可编程的器件通常只使用一些测试平台中固定的配置或一个外部文件来验证，并且大部分仿真一般只使用其中一种配置。

为什么不随机地产生器件的配置并下载到器件中去呢？自检验结构会使用器件配置描述符来根据目前的配置预测响应。同样，可以随机地生成测试平台的配置。例如，在验证以太网交换机时，为什么不随机地生成每个端口上的器件数目、速度和它们的MAC层地址呢？然后在随机生成的配置上使用功能覆盖测量，以确定哪些配置和配置参数的组合已经验证过了。

有必要的话约束配置

你的自检验结构是不是还不能支持所有可能的器件配置呢？或者，不能把所有可能的配置“编译”成写寄存器操作？或者，正在移植Verilog测试平台，这个测试平台只能通过\$readmemh

任务使用两种配置？这些都没有问题。在配置发生器上简单地加上约束就可以生成目前支持的配置。一旦能够支持其他的配置，再把相应的约束去除。

3.6.5 直接的测试用例

确定可能性低的测试用例

有一些功能用随机激励来验证的可能性很低。例如，在验证中断位是可屏蔽的时，需要把屏蔽位随机地设为1和0，而相应的中断位是1和0（比如，对所有的中断位，把中断位的交叉覆盖用屏蔽位的值填充）。中断通常是触发设计中异常事件的信号，使用随机激励很可能需要相当长的时间才能完成这个功能的验证。这些功能使用直接的测试可能更容易验证。

可以使用约束实现

编写直接的测试用例并不意味着要使用直接的过程实现。如果随机发生器被设计成高度可约束的，就可能对它加很多约束以生成直接的激励。例如，为了验证特定中断的可屏蔽能力，要约束CPU周期发生器，使它在中断屏蔽寄存器地址的适当位置写一个0，然后写一个1。然后可能要约束数据发生器来触发一个中断条件。一旦检测到中断条件，约束CPU发生器使它在屏蔽位再次写入一个0和一个1。用一个断言可以验证，外部中断只有在中断条件没有被屏蔽时才会发生。然而，如果最有效的方法是编写直接过程的测试用例，那就要放弃随机的环境，使得可以访问总线功能模型的事务层。

第一个测试用例是最简单的也是最困难的

当一个验证团队第一次接手一个新版本的设计时，要先完成一些简单的测试用例。这些测试用例的目标是验证正常工作时的基本功能。一旦通过了这些测试用例的最初调试，就可开始大量彻底地验证设计。

这些最初的调试测试用例虽然简单，但却是最难通过的。可能要花几星期时间运行这些简单测试。因为它们应用在不成熟的代码上，会碰到大量错误。这个前期的调试测试用例包括进行连续的写、读周期，或者传输一个路由包，或者执行一些直接的指令。

用于调试的测试用例可以是随机的

由于简单，你可能倾向于把最初的调试测试用例写成直接的测试用例。如果有一个设计良好的发生器，那么写成受约束的测试会简单得多。例如，把测试约束为两个周期，第一个周期为写周期，第二个周期为读周期，它们的地址是相同的。或者，约束路由包发生器为整个仿真只生成一个路由包。或者，约束指令生成器，让它只生成一个没有分支的算术指令。一旦最初的调试测试用例成功地通过了，就可以去除这些约束，建立随机的环境。

3.7 小结

写一个验证计划。它是对所有测试用例和所支持的测试平台功能的验证规范。实现和验证源自共同的规范。不要去验证一个实现。

定义验证设计所用的不同层次的粒度：模块、单元、组件、FPGA、ASIC、子系统、板级系统。为了更少地测试平台和更高集成度的测试，要在可观测性和可控制性之间做出平衡。

定义能够检测错误的自检验策略。

从设计规范确定功能，并列举哪些功能要验证。

在设计阶段就要考虑验证。构建设计使之尽可能易于验证。

如果测试用例的数量很少，可以使用直接的测试平台方法。对于每个功能，使用一个测试用例。用单独的测试平台实现每个测试用例。

从列举的功能里定义一个功能覆盖模型。从那些相同的功能里确定生成器所需的自由度和约束度，以便生成验证每个功能所要的激励。